



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that **Palak Chanchlani** of **D20A** semester **VIII**, have successfully completed necessary experiments in the **ITC801: Blockchain & DLT Lab** under my supervision in **VES Institute of Technology** during the academic year **2025-2026**.

Subject Teacher

Name: Mrs. Kajal Joseph

Signature:

Head of Department

Name: Dr. Mrs. Shalu Chopra

Signature:

Lab Teacher

Name: Kajal Joseph

Signature:



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

Lab Code	Lab Name	Credit
ITC801	Blockchain and DLT Lab	1

Programme Outcomes: The graduate will be able to:

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design / develop systems /components / processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.
PSO 1	Computing & Emerging Technology: Apply core computing and modern technologies, such as AI, Cloud Computing, IoT, cybersecurity, and data-driven tools, to build efficient and innovative solutions.
PSO 2	Professionalism & Innovation : Demonstrate ethics, teamwork, and an entrepreneurial mindset to deliver sustainable solutions for society.

Course Outcomes: Student will be able	
ITC801.1	Describe the basic concept of Blockchain and Distributed Ledger Technology.
ITC801.2	Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions
ITC801.3	Implement smart contracts in Ethereum using different development frameworks.
ITC801.4	Develop applications in permissioned Hyperledger Fabric network.
ITC801.5	Interpret different Crypto assets and Crypto currencies
ITC801.6	The use of Blockchain with AI, IoT and Cyber Security using case studies.

ITC801 Blockchain and DLT Lab													
CO	PO											PSO	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2
ITC801.1	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.2	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.3	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.4	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.5	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.6	2	3	3	3	3	3	3	3	3	2	2	3	3
Avg PO AL	2	3	3	3	3	3	3	3	3	2	2	3	3



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801 Blockchain and DLT Lab												
CO	Experiment											
	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9	Exp 10	Exp 11	Exp 12
ITC801.1	3	3	2	-	2	2	2	-	-	3	-	3
ITC801.2	2	3	3	-	2	2	3	2	-	3	-	3
ITC801.3	-	-	2	3	3	3	3	3	2	-	-	3
ITC801.4	-	-	-	-	-	-	-	-	-	-	3	3
ITC801.5	2	3	3	-	-	2	2	2	-	3	-	3
ITC801.6	-	-	-	-	-	-	-	-	-	2	-	3

INDEX

Sr. No.	List of Experiments
1	Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash
2	Create a Blockchain using Python
3	Create a Cryptocurrency using Python and perform mining in the Blockchain created.
4	Hands-on Solidity Programming Assignments for creating Smart Contracts
5	Deploying a Voting/Ballot Smart Contract
6	Creating Smart Contracts and performing transactions using Solidity and Remix IDE
7	Implement the embedding wallet (Metamask) and transaction using Solidity
8	To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

9	To implement a Private Ethereum Blockchain using Geth.
10	Explore the Cryptocurrency Landscape
11	Implementation of Hyperledger Fabric Network and Asset Management using Chaincode
12	Mini Project
12	Assignment-1
13	Assignment-2
14	Assignment-3
15	Mock Viva

Subject teacher



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment No. 01

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Roll No.	12
Name	Palak Chanchlani
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

NAME : Palak Chanchlani

ROLL NO: 05

CLASS : D20A

BATCH : C

BLOCKCHAIN EXP 1

AIM: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

THEORY:

Cryptographic Hash Function in Blockchain

A cryptographic hash function is a very important concept used in blockchain technology to provide security and data integrity. It is a mathematical algorithm that converts input data of any size into a fixed-length output called a **hash value**.

In blockchain systems, the most commonly used cryptographic hash function is **SHA-256 (Secure Hash Algorithm – 256 bits)**. No matter how small or large the input is, SHA-256 always generates a 256-bit hash value.

One of the main features of a cryptographic hash function is that it is **one-way** in nature. This means once the hash is generated, it is practically impossible to get back the original input data from the hash. This property ensures high security.

Cryptographic hash functions have the following important properties:

- **Deterministic:** The same input will always produce the same hash value.
- **Fast Computation:** Hash values can be generated quickly.
- **Avalanche Effect:** Even a small change in input results in a completely different hash.
- **Collision Resistance:** It is extremely difficult to find two different inputs that generate the same hash.
- **Pre-image Resistance:** It is computationally infeasible to determine the original input from a given hash.

In blockchain, cryptographic hash functions are used to secure transaction data, link blocks together, validate proof-of-work, and maintain immutability of the blockchain.

Merkle Tree

A Merkle Tree is a special data structure used in blockchain to store and verify large numbers of transactions efficiently. It is also known as a **hash tree** because it uses cryptographic hash functions at every level of the tree.

In a Merkle Tree, each transaction is first converted into a hash. These hashes form the leaf nodes of the tree. Then pairs of hashes are combined and hashed again to form higher-level nodes. This process continues until a single hash is produced at the top, known as the **Merkle Root**.

Merkle Trees help blockchain systems verify transaction data quickly without having to process every transaction individually. If even one transaction is altered, the hash of that transaction changes, which affects all the parent hashes and finally changes the Merkle Root.

Structure of Merkle Tree

The structure of a Merkle Tree consists of multiple hierarchical levels:

1. **Leaf Nodes**

These are the bottom-most nodes of the tree. Each leaf node contains the hash of an individual transaction.

2. **Intermediate Nodes**

These nodes are formed by combining two child node hashes and applying a hash function on the concatenated value.

3. **Root Node**

The top-most node is called the Merkle Root. It represents all transactions present in the block.

If the number of transactions is odd, the last transaction hash is duplicated to maintain a balanced binary tree structure. This ensures proper tree formation.

Merkle Root

The Merkle Root is the final hash obtained after recursively hashing all transaction hashes. It acts as a unique digital fingerprint for all transactions in a block.

The Merkle Root is stored in the **block header** of the blockchain. Any change in even a single transaction will result in a different Merkle Root. Therefore, the Merkle Root plays a crucial role in ensuring transaction integrity and security.

Working of Merkle Tree

The working of a Merkle Tree can be explained in the following steps:

1. Each transaction in a block is hashed using a cryptographic hash function such as SHA-256.
2. Two transaction hashes are paired and concatenated.
3. The concatenated value is hashed again to form a parent node.
4. This pairing and hashing process continues level by level.
5. When only one hash remains, it becomes the Merkle Root.

This hierarchical hashing structure allows blockchain nodes to verify transactions efficiently and securely.

Benefits of Merkle Tree

Merkle Trees provide several advantages in blockchain technology:

- Efficient verification of large data sets
- Faster transaction validation
- Reduced storage requirements
- High level of data security
- Easy detection of data tampering
- Supports lightweight and mobile blockchain clients

Use of Merkle Tree in Blockchain

In blockchain systems, Merkle Trees are used to store and verify transaction data inside a block. They enable nodes to verify a transaction without downloading the entire block. Merkle Trees are also used in **Simplified Payment Verification (SPV)**, which allows lightweight clients to verify transactions using only block headers and Merkle proofs.

Use Cases of Merkle Tree

Merkle Trees are widely used in many applications such as:

- Blockchain and cryptocurrency networks
- Distributed and decentralized systems
- Secure data storage and retrieval
- Digital signatures and authentication systems
- Version control systems
- Data integrity verification mechanisms

CODE AND OUTPUT :

1. Hash Generation using SHA-256: Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

Objective:

To generate a SHA-256 hash for a given input string.

```
[3]
✓ 12s  import hashlib

data = input("Enter a string to hash: ")

hash_object = hashlib.sha256(data.encode())
hash_value = hash_object.hexdigest()

print("SHA-256 Hash:", hash_value)

▼ ... Enter a string to hash: palak
SHA-256 Hash: 34f3c63faab4e96fc144b8bb23f493ea9356cb8220845ee1ce71aced0b36ba88
```

2. Target Hash Generation with Nonce: Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

Objective:

To generate a hash by combining input data with a nonce value.

```
[4]
✓ 17s  import hashlib

data = input("Enter data: ")
nonce = input("Enter nonce value: ")

combined_data = data + nonce
hash_value = hashlib.sha256(combined_data.encode()).hexdigest()

print("Combined Data:", combined_data)
print("Generated Hash:", hash_value)

▼ ... Enter data: Palak
Enter nonce value: 12
Combined Data: Palak12
Generated Hash: 7cb94bad0a68b7f0a3c2c05e9f5e78b213e4f12fb56bd5ebc3bf30d651f1849f
```

3. Proof-of-Work Puzzle Solving: Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

Objective:

To simulate blockchain mining by finding a nonce that produces a hash with leading zeros.

```
[5]
✓ 29s  import hashlib

data = input("Enter block data: ")
difficulty = int(input("Enter difficulty (number of leading zeros): "))

prefix = '0' * difficulty
nonce = 0

while True:
    text = data + str(nonce)
    hash_value = hashlib.sha256(text.encode()).hexdigest()

    if hash_value.startswith(prefix):
        print("Nonce found:", nonce)
        print("Hash:", hash_value)
        break
    nonce += 1
```

▼ ... Enter block data: 12
Enter difficulty (number of leading zeros): 5
Nonce found: 396064
Hash: 00000bdebc0a945fcd1a1b6127267393dfc299c89b031b7879d926a32921e9f6

4. Merkle Tree Construction: Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Objective

To construct a Merkle Tree from a list of transactions and generate the Merkle Root.

```
[2]
✓ 12s  import hashlib

def sha256(data):
    return hashlib.sha256(data.encode()).hexdigest()

def merkle_root(transactions):
    hashes = [sha256(tx) for tx in transactions]

    while len(hashes) > 1:
        if len(hashes) % 2 != 0:
            hashes.append(hashes[-1])    # duplicate last hash if odd

        new_level = []
        for i in range(0, len(hashes), 2):
            combined = hashes[i] + hashes[i+1]
            new_level.append(sha256(combined))

        hashes = new_level

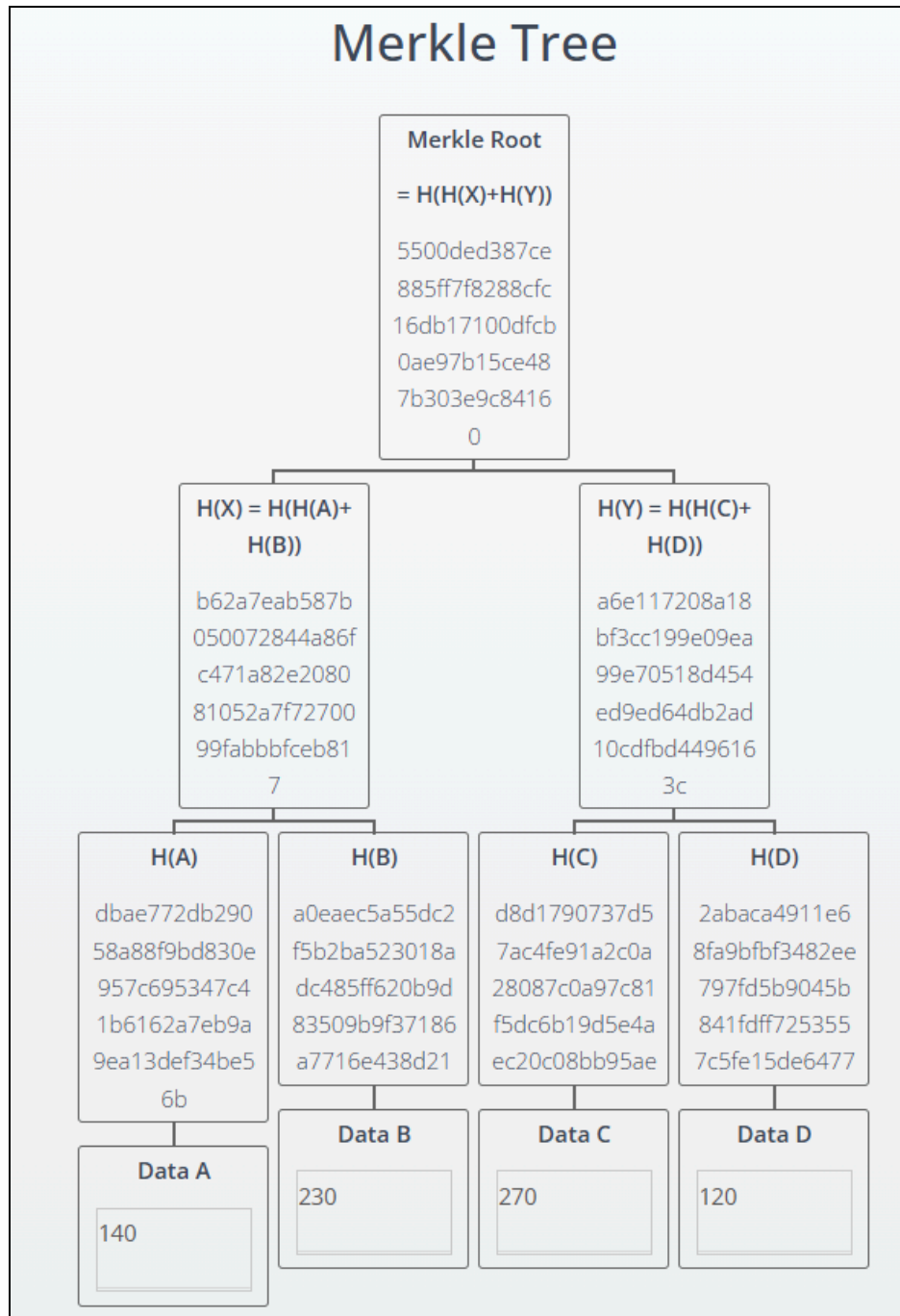
    return hashes[0]

# User input
n = int(input("Enter number of transactions: "))
transactions = []

for i in range(n):
    tx = input(f"Enter transaction {i+1}: ")
    transactions.append(tx)

root = merkle_root(transactions)
print("Merkle Root Hash:", root)

▼ ... Enter number of transactions: 3
Enter transaction 1: 23
Enter transaction 2: 27
Enter transaction 3: 12
Merkle Root Hash: bed6300e5ccc61db316e49f55e512df3e62d7f06ac646636a524188120b2fdfa
```

Conclusion:

In this experiment, we studied cryptographic hash functions and their importance in blockchain security using SHA-256. We also understood the concept and working of Merkle Trees for efficient transaction verification. The Merkle Root ensures data integrity by detecting any changes in transactions, making blockchain systems secure and trustworthy.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain & DLT Lab
Experiment 02

Aim: Create a Blockchain using Python

Roll No.	12
Name	Palak Chanchlani
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

NAME : Palak Chanchlani

ROLL NO: 05

CLASS : D20A

BATCH : C

BLOCKCHAIN EXP 2

Aim: Create a Blockchain using Python

1. What is blockchain?

Blockchain is a digital record-keeping system where information is stored in connected blocks. Each block is linked to the previous one using cryptographic hashes, forming a secure chain.

- It works like a distributed ledger, shared across many computers (nodes).
- Once data is added, it becomes permanent and cannot be changed without breaking the chain.
- Every participant in the network has a copy of the blockchain, ensuring transparency.
- Consensus mechanisms (like Proof of Work) make sure only valid data is added.
- Because of decentralization, no single authority controls the blockchain, making it trustworthy and tamper-resistant.

2. What is a block?

A block is the basic unit of a blockchain. It acts like a container that holds verified data.

Main parts of a block:

- **Index:** Position of the block in the chain.
- **Timestamp:** Exact time when the block was created.
- **Proof:** Result of solving the cryptographic puzzle.
- **Hash:** Unique digital fingerprint of the block.
- **Previous Hash:** Connects the block to the one before it, ensuring continuity.

In simple terms, a block is like a “page” in a digital notebook, where each page is permanently linked to the previous one.

3. Process of mining

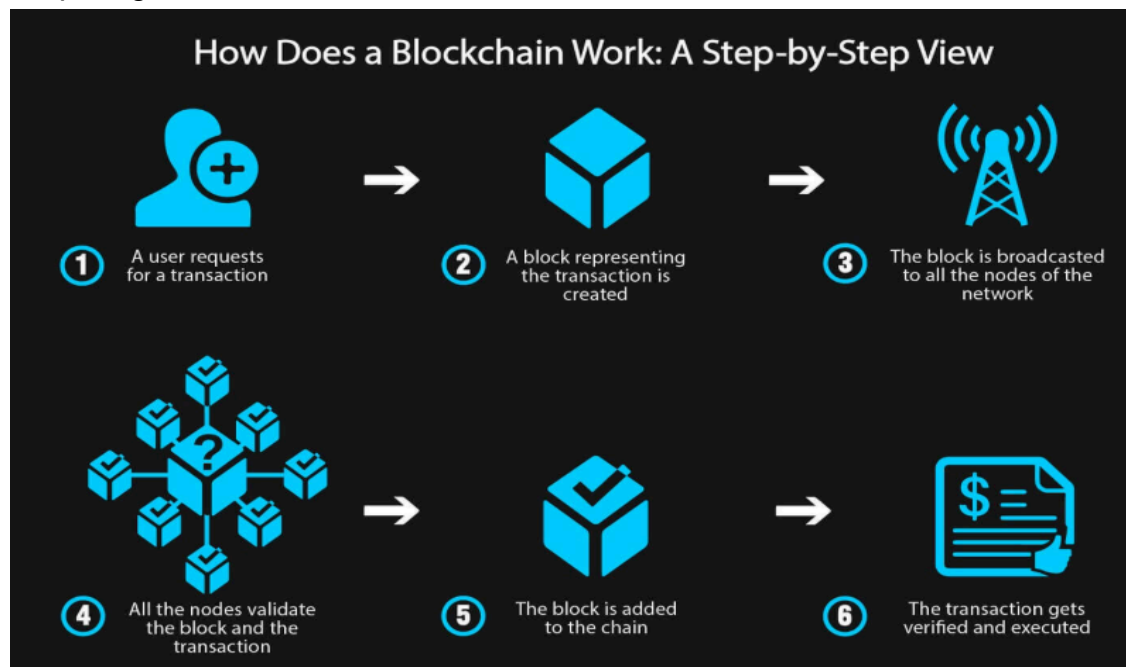
Mining is the method of adding new blocks to the blockchain by solving a puzzle.

Steps in mining:

1. **Transaction broadcast:** Users send transactions to the network.
2. **Block formation:** Miners collect these transactions into a candidate block.

3. **Proof of Work:** Miners try different numbers (nonce) until the hash of the block starts with a fixed number of zeros.
4. **Block validation:** The first miner to solve the puzzle shares the solution, and other nodes verify it.
5. **Block addition:** Once verified, the block is added permanently to the chain.
6. **Reward:** The miner receives incentives (like cryptocurrency or recognition).

Mining ensures that adding blocks requires effort, which protects the blockchain from tampering.



4. How to check validity of block in blockchain

To confirm that a blockchain is valid, each block must pass certain checks:

- **Hash linkage:** The previous_hash of a block must match the actual hash of the block before it.
- **Proof of Work check:** The block's proof must satisfy the condition (hash starting with zeros).
- **Transaction integrity:** All transactions inside the block must follow the rules of the network.
- **Chain consistency:** If any block is altered, its hash changes, breaking the chain and making it invalid.

In short, validity ensures that the blockchain has not been tampered with and all blocks are correctly linked.

Code :

```
# -----  
# Simple Blockchain Implementation using Python & Flask  
# -----
```

```
# Required installation:  
# pip install flask==2.2.5  
import datetime  
import hashlib  
import json  
from flask import Flask, jsonify
```

```
# -----  
# Blockchain Class Definition  
# -----  
class Blockchain:  
    def __init__(self):  
        self.chain = []  
        # Create the first block (Genesis Block)  
        self.create_block(proof=1, previous_hash='0')  
  
    def create_block(self, proof, previous_hash):  
        block = {  
            'index': len(self.chain) + 1,  
            'timestamp': str(datetime.datetime.now()),  
            'proof': proof,  
            'previous_hash': previous_hash  
        }  
        self.chain.append(block)  
        return block  
  
    def get_previous_block(self):  
        return self.chain[-1]  
  
    def proof_of_work(self, previous_proof):  
        new_proof = 1  
        while True:  
            hash_value = hashlib.sha256(  
                str(new_proof**2 - previous_proof**2).encode()  
            ).hexdigest()  
            if hash_value[:4] == '0000':  
                return new_proof  
            new_proof += 1
```

```

def hash(self, block):
    encoded_block = json.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    index = 1

    while index < len(chain):
        current_block = chain[index]

        # Check if previous hash matches
        if current_block['previous_hash'] != self.hash(previous_block):
            return False

        # Check Proof of Work validity
        previous_proof = previous_block['proof']
        proof = current_block['proof']
        hash_value = hashlib.sha256(
            str(proof**2 - previous_proof**2).encode()
        ).hexdigest()
        if hash_value[:4] != '0000':
            return False
        previous_block = current_block
        index += 1
    return True

# -----
# Flask Application Setup
# -----
app = Flask(__name__)
blockchain = Blockchain()

# Home route (to avoid 404 at root URL)
@app.route('/', methods=['GET'])
def home():
    return "Welcome to Blockchain Lab! Use /mine_block, /get_chain, /is_valid"

@app.route('/mine_block', methods=['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    block = blockchain.create_block(proof, previous_hash)

```

```

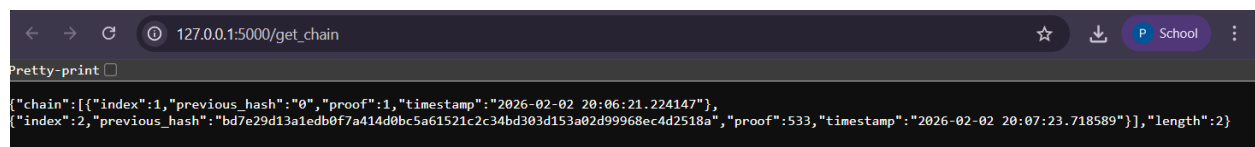
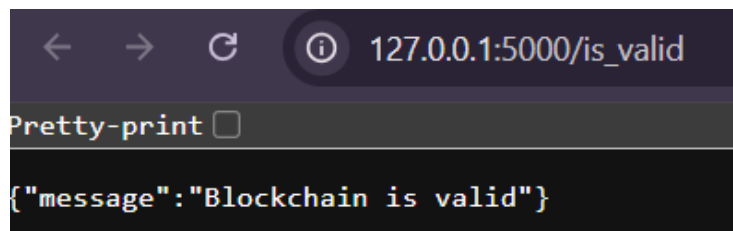
response = {
    'message': 'New block has been successfully mined',
    'index': block['index'],
    'timestamp': block['timestamp'],
    'proof': block['proof'],
    'previous_hash': block['previous_hash']
}
return jsonify(response), 200

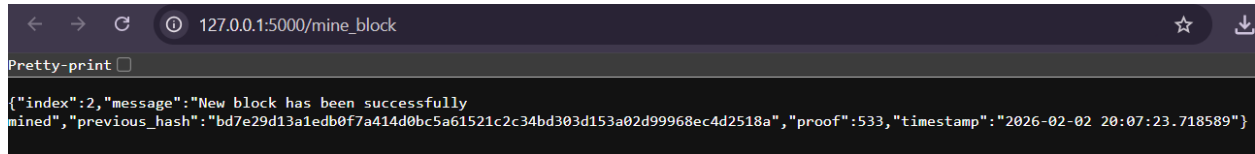
@app.route('/get_chain', methods=['GET'])
def get_chain():
    return jsonify({
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }), 200

@app.route('/is_valid', methods=['GET'])
def is_valid():
    result = blockchain.is_chain_valid(blockchain.chain)
    if result:
        return jsonify({'message': 'Blockchain is valid'}), 200
    else:
        return jsonify({'message': 'Blockchain is invalid'}), 200
# -----
# Run the server
# -----
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

Output :



A screenshot of a web browser window. The address bar shows the URL '127.0.0.1:5000/mine_block'. Below the address bar, there is a 'Pretty-print' button. The main content area displays a JSON object:

```
{ "index": 2, "message": "New block has been successfully mined", "previous_hash": "bd7e29d13a1edb0f7a414d0bc5a61521c2c34bd303d153a02d99968ec4d2518a", "proof": 533, "timestamp": "2026-02-02 20:07:23.718589" }
```

Conclusion:

In this experiment, we created a simple blockchain using Python and Flask. Each block was linked using cryptographic hashing, and Proof of Work was used to secure the chain. By checking validity, we ensured that the blockchain remains consistent and trustworthy. This experiment demonstrates the core principles of blockchain — immutability, decentralization, and security — in a simplified environment.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Roll No.	12
Name	Palak Chanchlani
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3: Implement smart contracts in Ethereum using different development frameworks. CO5: Interpret different Crypto assets and Crypto currencies

Name: Palak Chanchlani
Class: D20A
Roll No: 12

EXPERIMENT NO - 3

AIM

To design and implement a cryptocurrency using Python and demonstrate the mining process in a blockchain environment.

THEORY

1. Blockchain Overview

Blockchain is a decentralized and distributed digital ledger used to securely record transactions across multiple computers called nodes. Unlike traditional systems, it does not rely on a central authority. Instead, every node in the network stores and maintains an identical copy of the ledger.

Blocks are linked together using cryptographic hash values, forming a secure and continuous chain. Each block depends on the hash of the previous block, so even a small change in data will modify the hash and break the chain. This makes unauthorized changes easy to detect.

Because of this structure, blockchain ensures data integrity, transparency, and immutability, meaning once data is recorded, it cannot be altered.

- **Transactions:** A group of verified records representing the transfer of data or digital assets between participants.
- **Timestamp:** Stores the exact date and time when a block is created, helping maintain the correct order of blocks.
- **Previous hash:** A reference to the hash of the earlier block, which securely links blocks together.
- **Current block hash:** A unique cryptographic value used to identify the block and verify that its data remains unchanged.

2. Mining

Mining is the process through which new blocks are added to the blockchain.

It includes the following steps:

- Collecting pending transactions
- Creating a block header
- Solving a Proof-of-Work (PoW) problem

Collecting pending transactions

Before a new block is created, all unconfirmed transactions in the network are collected. These transactions are checked for validity and then prepared to be added to the next block.

Creating a block header

After selecting the transactions, a block header is created. It contains important details such as the previous block's hash, timestamp, nonce, and Merkle root, which together define the structure of the block.

Solving Proof-of-Work (PoW)

In this step, miners solve a complex mathematical problem by repeatedly changing a nonce value until a valid hash that meets the difficulty requirement is found. Successfully solving this problem allows the block to be added to the blockchain.

Mining involves competition among miners, and the first miner to find a valid hash adds the block to the chain and receives a reward.

3. Multi-Node Blockchain Network

In this experiment, a multi-node blockchain network is simulated using three nodes running on different ports (5001, 5002, and 5003).

Each node:

- Operates independently
- Maintains its own copy of the blockchain
- Communicates with other nodes to share updates

Operates independently

Each node functions on its own and validates data without depending on any central authority.

Maintains its own copy of the blockchain

Every node stores a complete copy of the blockchain, which improves security and ensures fault tolerance.

Communicates with peer nodes

Nodes exchange newly mined blocks and updates with each other to keep the entire network consistent.

Overall, each node works independently while continuously sharing information with other nodes, ensuring synchronization and reliability across the network.

4. Consensus Mechanism

Consensus is a method used to achieve agreement among all nodes in a blockchain network.

Key points of consensus:

- Ensures all nodes have the same and correct version of the blockchain
- Removes the need for a central authority
- Prevents double spending and fraudulent activities
- Uses algorithms such as Proof-of-Work and the Longest Chain Rule
- Maintains trust, security, and synchronization in the network

A consensus mechanism allows nodes to independently verify transactions and blocks before accepting them. It resolves conflicts between different blockchain versions and ensures that only valid data is added, keeping the decentralized system secure and consistent.

5. Transactions and Mining Reward

A transaction in a cryptocurrency system includes:

- Sender address: Identifies the account sending the cryptocurrency
- Receiver address: Identifies the account receiving the cryptocurrency
- Transaction amount: Specifies how much cryptocurrency is being transferred

When a miner successfully adds a new block to the blockchain, they receive a mining reward for their effort.

6. Chain Replacement

Chain replacement is used to maintain consistency across the blockchain network.

When the `/replace_chain` endpoint is activated:

- A node requests blockchain data from other nodes

- The received chains are validated
- The node replaces its own chain if a longer and valid chain is found

This process helps resolve conflicts between different versions of the blockchain and ensures that all nodes remain synchronized.

CODE :

```
class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
        self.create_block(proof=1, previous_hash='0')
        self.nodes = set()

    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,

            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash,
            'transactions': self.transactions
        }
        self.transactions = []
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False
        while not check_proof:
            hash_operation = hashlib.sha256(
                str(new_proof**2 - previous_proof**2).encode()
            ).hexdigest()
            if hash_operation[:4] == '0000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof
```

```

def hash(self, block):
    encoded_block = json.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        if block['previous_hash'] != self.hash(previous_block):
            return False
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(
            str(proof**2 - previous_proof**2).encode()

        ).hexdigest()
        if hash_operation[:4] != '0000':
            return False
        previous_block = block
        block_index += 1
    return True

def add_transaction(self, sender, receiver, amount):
    self.transactions.append({
        'sender': sender,
        'receiver': receiver,
        'amount': amount
    })
    previous_block = self.get_previous_block()
    return previous_block['index'] + 1

def add_node(self, address):
    parsed_url = urlparse(address)
    self.nodes.add(parsed_url.netloc)

```

```

def replace_chain(self):
    network = self.nodes
    longest_chain = None
    max_length = len(self.chain)
    for node in network:
        response = requests.get(f'http://{node}/get_chain')
        if response.status_code == 200:
            length = response.json()['length']
            chain = response.json()['chain']
            if length > max_length and self.is_chain_valid(chain):
                max_length = length
                longest_chain = chain
    if longest_chain:
        self.chain = longest_chain
        return True
    return False

app = Flask(__name__)

node_address = str(uuid4()).replace('-', '')

```

```

blockchain = Blockchain()

@app.route('/mine_block', methods=['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    blockchain.add_transaction(
        sender=node_address,
        receiver='Richard',
        amount=1
    )
    block = blockchain.create_block(proof, previous_hash)
    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions']
    }
    return jsonify(response), 200

@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }
    return jsonify(response), 200

@app.route('/is_valid', methods=['GET'])
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
        response = {'message': 'The Blockchain is not valid.'}

```

```

        return jsonify(response), 200

@app.route('/add_transaction', methods=['POST'])
def add_transaction():
    json_data = request.get_json()
    transaction_keys = ['sender', 'receiver', 'amount']
    if not all(key in json_data for key in transaction_keys):
        return 'Some elements of the transaction are missing', 400
    index = blockchain.add_transaction(
        json_data['sender'],
        json_data['receiver'],
        json_data['amount']
    )
    response = {
        'message': f'This transaction will be added to Block {index}'
    }
    return jsonify(response), 201

@app.route('/connect_node', methods=['POST'])
def connect_node():
    json_data = request.get_json()
    nodes = json_data.get('nodes')
    if nodes is None:
        return "No node", 400
    for node in nodes:
        blockchain.add_node(node)
    response = {
        'message': 'All the nodes are now connected.',
        'total_nodes': list(blockchain.nodes)
    }
    return jsonify(response), 201

@app.route('/replace_chain', methods=['GET'])
def replace_chain():
    is_chain_replaced = blockchain.replace_chain()
    if is_chain_replaced:
        response = {
            'message': 'The chain was replaced by the longest one.',
            'new_chain': blockchain.chain
        }
    else:
        response = {
            'message': 'The chain is already the largest one.',
            'actual_chain': blockchain.chain
        }
    return jsonify(response), 200

app.run(host='0.0.0.0', port=5000)

```

OUTPUT :

GET echo GET

Anchor tags in Postman documentation / GET Requests / echo GET

GET http://127.0.0.1:5000/mine_block

Overview Params Authorization **Headers (7)** Body Scripts Settings

Headers 7 hidden

Key	Value

Body Cookies Headers (5) Test Results

{ } JSON Preview Visualize

```
1 {
2   "block": {
3     "index": 3,
4     "previous_hash": "4d56ca2f3e090aa136cbc8e9c40c5b4154438c7ed33a85fd9461e8c53d2afe04",
5     "proof": 45293,
6     "timestamp": "2026-02-03 09:29:02.261406",
7     "transactions": [
8       {
9         "amount": 1,
10        "receiver": "Miner",
11        "sender": "f50c937a472445029581f4c4dd5626fd"
12      }
13    ]
14  },
15  "message": "Block mined successfully"
16 }
```

GET echo GET

Anchor tags in Postman documentation / GET Requests / echo GET

GET http://127.0.0.1:5000/get_chain

Overview Params Authorization **Headers (7)** Body Scripts Settings

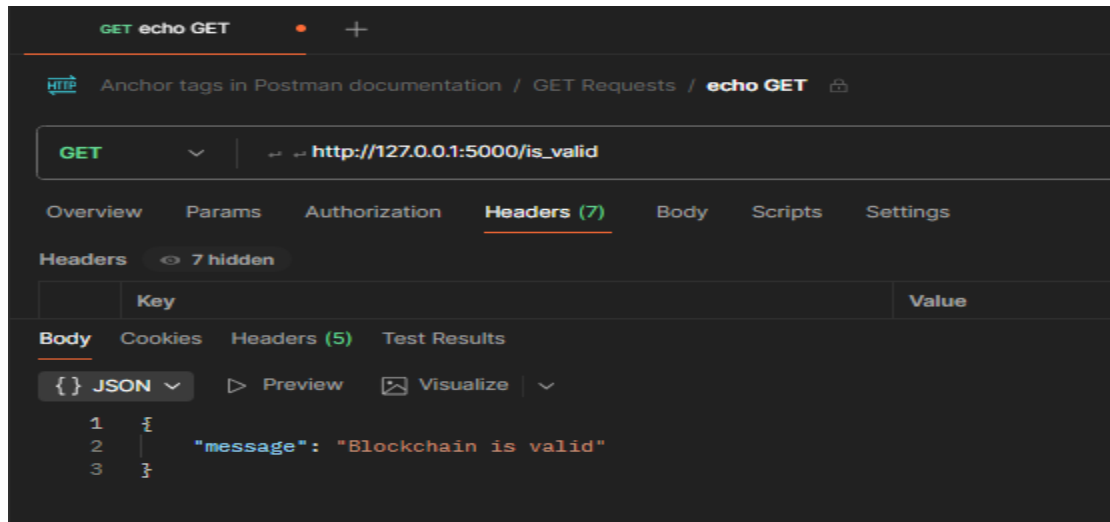
Headers 7 hidden

Key	Value

Body Cookies Headers (5) Test Results

{ } JSON Preview Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-02-03 09:19:02.441105",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "badca82819f0e8ae886535c28b2aef8affbd61def049537257cb84f22913acd7",
13      "proof": 533,
14      "timestamp": "2026-02-03 09:28:30.853839",
15      "transactions": [
16        {
17          "amount": 1,
18          "receiver": "Miner",
19          "sender": "f50c937a472445029581f4c4dd5626fd"
20        }
21      ]
22    },
23  ]
24 }
```



Conclusion:

This implementation demonstrates how blockchain technology works by creating a decentralized ledger using Python. The code shows how blocks are securely linked using cryptographic hashes, how transactions are recorded, and how Proof-of-Work is used to mine new blocks. The use of consensus through chain validation and replacement ensures data integrity and synchronization across nodes. Overall, the system highlights the core principles of blockchain such as decentralization, transparency, security, and immutability.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 04

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Roll No.	12
Name	Palak Chanchlani
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Name: Palak Chanchlani

Class: D20A

Roll No: 12

Experiment 4

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Theory

1. Introduction to Solidity and Smart Contracts

Solidity is a contract-oriented programming language designed for Ethereum. It is high-level, statically typed, and strongly influenced by languages like JavaScript, Python, and C++. Smart contracts written in Solidity allow developers to encode business logic directly on the blockchain, ensuring transparency, immutability, and automation.

Primitive Data Types:

- **uint / int:** Represent unsigned and signed integers of different bit sizes. For example, uint256 is widely used in token contracts to store balances, while int128 may be used in mathematical models.
- **bool:** Stores logical values (true or false). Often used in conditions such as verifying ownership or validating transactions.
- **address:** Holds Ethereum account or contract addresses (20 bytes). This type is essential for identifying participants and enabling contract-to-contract communication.
- **string / bytes:** Used for textual and binary data. Strings are suitable for names or metadata, while bytes are efficient for raw data like cryptographic hashes.

Variable Categories:

- **State Variables:** Permanently stored on-chain. Example: total supply of a token.
- **Local Variables:** Exist only during function execution and vanish afterward.
- **Global Variables:** Predefined values such as msg.sender, msg.value, and block.timestamp that provide transaction context.

Function Types:

- **pure:** Functions that neither read nor modify blockchain state. Example: a function that calculates square of a number.
- **view:** Functions that can read state variables but cannot alter them. Example: fetching a stored balance.

2. Inputs and Outputs in Functions

Functions in Solidity can accept parameters and return values, making contracts interactive.

- **Inputs:** Allow external users or contracts to pass data. Example: a deposit function accepting Ether.
- **Outputs:** Provide results back to the caller. Example: a withdrawal function returning a boolean.
- **Named Returns:** Improve readability and debugging by explicitly naming return variables.

3. Visibility, Modifiers, and Constructors

Visibility:

- **public:** Accessible internally and externally.
- **private:** Restricted to the same contract.
- **internal:** Available within the contract and derived contracts.
- **external:** Callable only from outside.

Modifiers: Reusable logic that alters function behaviour.

Example:

```
modifier onlyOwner {  
    require(msg.sender == owner, "Not authorized");  
}
```

Constructor: Special function executed once during deployment. Typically used to initialize critical variables, such as setting the deployer as the contract owner.

4. Control Flow

Solidity supports conditional and iterative structures:

- **if-else:** Enables decision-making, such as checking sufficient balance before transfer.
- **Loops (for, while, do-while):** Allow repetitive tasks like iterating through arrays. Must be used cautiously to avoid excessive gas consumption.

5. Data Structures

- **Arrays:** Ordered collections of elements. Example: storing participant addresses.
- **Mappings:** Key-value pairs for fast lookups.
Example: mapping(address => uint) for balances.
- **Structs:** Group related attributes.

Example:

```
struct Player {  
    string name;  
    uint score;  
}
```

- **Enums:** Define a set of constant values for clarity. Example:
`enum Status { Pending, Active, Closed }`

6. Data Locations

Solidity defines three primary data locations:

- **storage:** Permanent, on-chain data (state variables).
- **memory:** Temporary data during function execution.
- **calldata:** Non-modifiable, efficient storage for external function inputs.

Efficient use of data locations reduces gas costs and improves performance.

7. Transactions: Ether, Wei, Gas, and Transfers

Ether and Wei: Ether is Ethereum's native currency. 1 Ether = (10^{18}) Wei. This precision is vital for financial transactions.

Gas and Gas Price:

- Gas represents computational effort.
- Gas price determines transaction speed and cost. Higher gas prices incentivize miners to process transactions faster.

Sending Transactions:

- **transfer():** Safe, limited gas forwarding.
- **send():** Returns a boolean indicating success or failure.
- **call():** Flexible and widely used in modern Solidity development.

Output -

1.introduction.sol

The screenshot shows the LEARNETH IDE interface. On the left, the '1. Introduction' tutorial is displayed, explaining the purpose of the counter contract and the roles of various keywords like `contract`, `uint`, `public`, `view`, `inc`, and `dec`. On the right, the `introduction.sol` file is open, showing Solidity code for a contract named `MyContract` with a `name` variable and a `constructor` that sets `name` to "Alice".

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract MyContract {
5     string public name;
6
7     constructor() {
8         name = "Alice";
9     }
10 }
11
```

2. basicSyntax.sol

The screenshot shows the LEARNETH IDE interface. On the left, the '2. Basic Syntax' tutorial is displayed, explaining the structure of a smart contract, including the license, Solidity version, contract definition, state variable, and visibility. On the right, the `basicSyntax.sol` file is open, showing the Solidity code for a contract named `MyContract` with a `greet` state variable of type `string`.

```
1 pragma solidity ^0.8.3;
2
3 contract MyContract {
4     string public greet = "Hello World!";
5 }
6
```

3. primitiveDataTypes.sol

LEARNETH

Tutorials list

Syllabus

3. Primitive Data Types

3 / 19

3. Primitive Data Types

In this section, we will show you Solidity's primitive data types, how to declare them, and their characteristics.

bool

You can declare data a boolean type by using the keyword 'bool'. Booleans can either have the value `true` or `false`.

uint

We use the keywords `uint` and `uint8` to `uint256` to declare an *unsigned integer type* (they don't have a sign, unlike -12, for example). Uints are integers that are positive or zero and range from 8 bits to 256 bits. The type `uint` is the same as `uint256`.

int

We use the keywords `int` and `int8` to `int256` to declare an integer type. Integers can be positive, negative, or zero and range from 8 bits to 256 bits. The type `int` is the same as `int256`.

address

Variables of the type `address` hold a 20-byte value, which is the size of an Ethereum address. There is also a special kind of Ethereum address, `address`.

Compile

introduction.sol

basicSyntax.sol 1

primitiveDataTypes.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.31;
3
4 contract Primitives {
5     address public addr = 0xab8483f64d9c6d1ecf9b849Ae677d03315835cb2;
6
7     // New public address (different from addr)
8     address public newAddr = 0x4B0897b0513fdC7C54186d9D7E929C4e5364D2d8;
9
10    // Public negative number
11    int public neg = -10;
12
13    // Smallest uint type with smallest value
14    uint8 public newU = 0;
15 }
16
```

4. variables.sol

LEARNETH

Tutorials list

Syllabus

4. Variables

4 / 19

Global variables, also called *special variables*, exist in the global namespace. They don't need to be declared but can be accessed from within your contract. Global Variables are used to retrieve information about the blockchain, particular addresses, contracts, and transactions.

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

introduction.sol

basicSyntax.sol 1

primitiveDataTypes.sol

variables.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.31;
3
4 contract Variables {
5
6     uint public blockNumber;
7
8     function doSomething() public { 22255 gas
9         blockNumber = block.number;
10     }
11 }
12
```


5.1 readAndWrite.sol

LEARNETH

Tutorials list

Syllabus

5.1 Functions - Reading and Writing to a State Variable

5 / 19

5.1 Functions - Reading and Writing to a State Variable

This section will give a short introduction to functions and teach you how to use them to read from and write to a state variable.

As in other languages, we use functions in Solidity to create modular, reusable code. However, Solidity functions have some particularities.

Solidity functions can be split into two types:

1. Functions that modify the state of the blockchain, like writing to a state variable. In this contract, the `set` function (line 9) changes the state variable `num`.
2. Functions that don't modify the state of the blockchain. These functions are marked `view` or `pure`. For example, in this contract, the `get` function (line 14) marked `view` that only returns `num` does not change the state.

To define a function, use the `function` keyword followed by a unique name.

If the function takes inputs like our `set` function (line 9), you must specify the parameter types and names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we

Compile

variables.sol

.prettierrc.json

readAndWrite.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.31;
3
4 contract SimpleStorage {
5
6     bool public b = true;
7
8     function get_b() public view returns (bool) { 2472 gas
9         return b;
10     }
11 }
12
```

5.2 viewAndPure.sol

LEARNETH

Tutorials list

Syllabus

5.2 Functions - View and Pure

6 / 19

5.2 Functions - View and Pure

This section will look into the types of functions that don't modify the state of the blockchain: *view* and *pure* functions.

View Functions

View functions promise to not modify the state.

"The following statements are considered modifying the state:

1. Writing to state variables.
2. Emitting events.
3. Creating other contracts.
4. Using `selfdestruct`.
5. Sending Ether via calls.
6. Calling any function not marked `view` or `pure`.
7. Using low-level calls.
8. Using inline assembly that contains certain opcodes."

From the [Solidity documentation](#).

You can declare a view function using the keyword `view`. In this contract, `addToX` (line 8) is a view function. This function takes the parameter `y` and returns the

Compile

variables.sol

.prettierrc.json

readAndWrite.sol

viewAndPure.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addToX(uint y) public view returns (uint) { infinite gas
9         return x + y;
10     }
11
12     // Promise not to modify or read from the state.
13     function add(uint i, uint j) public pure returns (uint) { infinite gas
14         return i + j;
15     }
16
17     function addToX2(uint y) public { infinite gas
18         x = x + y;
19     }
20 }
```

5.3 modifiersAndConstructor.sol

LEARNETH

Tutorials list

Syllabus

5.3 Functions - Modifiers and Constructors

7 / 19

A constructor function is executed upon the creation of a contract. You can use it to run contract initialization code. The constructor can have parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

★ Assignment

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that `x` can only be increased.
3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

pure.sol

viewAndPure_answer.sol

modifiersAndConstructors.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract FunctionModifier {
5     // We will use these variables to demonstrate how to use
6     // modifiers.
7     address public owner;
8     uint public x = 10;
9     bool public locked;
10
11     constructor() {
12         // Set the transaction sender as the owner of the contract.
13         owner = msg.sender;
14     }
15
16     // Modifier to check that the caller is the owner of
17     // the contract.
18     modifier onlyOwner() {
19         require(msg.sender == owner, "Not owner");
20         // Underscore is a special character only used inside
21         // a function modifier and it tells Solidity to
22         // execute the rest of the code.
23         _;
24     }
25
26     // Modifiers can take inputs. This modifier checks that the
27     // address passed in is not the zero address.
28     modifier validAddress(address _addr) {
29         require(_addr != address(0), "Not valid address");
30         _;
31     }
32 }
```

5.4 inputsAndOutputs.sol

LEARNETH

Tutorials list

Syllabus

5.4 Functions - Inputs and Outputs

8 / 19

5.4 Functions - Inputs and Outputs

In this section, we will learn more about the inputs and outputs of functions.

Multiple named Outputs

Functions can return multiple values that can be named and assigned to their name.

The `returnMany` function (line 6) shows how to return multiple values. You will often return multiple values. It could be a function that collects outputs of various functions and returns them in a single function call for example.

The `named` function (line 19) shows how to name return values. Naming return values helps with the readability of your contracts. Named return values make it easier to keep track of the values and the order in which they are returned. You can also assign values to a name.

The `assigned` function (line 33) shows how to assign values to a name. When you assign values to a name you can omit (leave out) the return statement and return them individually.

Deconstructing Assignments

You can use deconstructing assignments to unpack values into distinct variables.

The `destructingAssignments` function (line 49) assigns the values of the `returnMany`

Compiled

undPure.sol

viewAndPure_answer.sol

modifiersAndConstructors.sol

inputsAndOutputs.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Function {
5     // Functions can return multiple values.
6     function returnMany() infinite gas
7     {
8         public
9         pure
10         returns (
11             uint,
12             bool,
13             uint
14         )
15     {
16         return (1, true, 2);
17     }
18
19     // Return values can be named.
20     function named() infinite gas
21     {
22         public
23         pure
24         returns (
25             uint x,
26             bool b,
27             uint y
28         )
29     {
30         return (1, true, 2);
31     }
32
33     // Return values can be assigned to their name.
34     // In this case the return statement can be omitted.
```

6. visibility.sol

LEARNETH

Tutorials list

Syllabus

6. Visibility

9 / 19

6. Visibility

The `visibility` specifier is used to control who has access to functions and state variables.

There are four types of visibilities: `external`, `public`, `internal`, and `private`.

They regulate if functions and state variables can be called from inside the contract, from contracts that derive from the contract (child contracts), or from other contracts and transactions.

private

- Can be called from inside the contract

internal

- Can be called from inside the contract
- Can be called from a child contract

public

- Can be called from inside the contract
- Can be called from a child contract
- Can be called from other contracts or transactions

external

Compiled

viewAndPure_answer.sol

modifiersAndConstructors.sol

inputsAndOutputs.sol

visibility.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Base {
5     // Private function can only be called
6     // - inside this contract
7     // - Contracts that inherit this contract cannot call this function.
8     function privateFunc() private pure returns (string memory) {
9         return "private function called";
10    }
11
12    function testPrivateFunc() public pure returns (string memory) {
13        return privateFunc();
14    }
15
16    // Internal function can be called
17    // - inside this contract
18    // - inside contracts that inherit this contract
19    function internalFunc() internal pure returns (string memory) {
20        return "internal function called";
21    }
22
23    function testInternalFunc() public pure virtual returns (string memory) {
24        return internalFunc();
25    }
26
27    // Public functions can be called
28    // - inside this contract
29    // - inside contracts that inherit this contract
30    // - by other contracts and accounts
31    function publicFunc() public pure returns (string memory) {
32        return "public function called";
33    }
34}
```

7.1 ControlFlow.sol

LEARNETH

Tutorials list

Syllabus

7.1 Control Flow - If/Else

10 / 19

7.1 Control Flow - If/Else

In this contract, the `foo` function uses the `if/else` statement (line 10) to return `0` if none of the other conditions are met.

else if

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the `foo` function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

[Watch a video tutorial on the If/Else statement.](#)

★ **Assignment**

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

er.sol

modifiersAndConstructors.sol

inputsAndOutputs.sol

visibility.sol

IfElse.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract IfElse {
5     function foo(uint x) public pure returns (uint) {
6         if (x < 10) {
7             return 0;
8         } else if (x < 20) {
9             return 1;
10        } else {
11            return 2;
12        }
13    }
14
15    function ternary(uint x) public pure returns (uint) {
16        // if (x < 10) {
17        //     return 1;
18        // }
19        // return 2;
20
21        // shorthand way to write if / else statement
22        return x < 10 ? 1 : 2;
23    }
24
25    function evenCheck(uint y) public pure returns (bool) {
26        return y % 2 == 0 ? true : false;
27    }
28}
```

7.2 loops.sol

LEARNETH

Tutorials list

Syllabus

7.2 Control Flow - Loops

11 / 19

7.2 Control Flow - Loops

Solidity supports iterative control flow statements that allow contracts to execute code repeatedly.

Solidity differentiates between three types of loops: `for`, `while`, and `do while` loops.

for

Generally, `for` loops (line 7) are great if you know how many times you want to execute a certain block of code. In solidity, you should specify this amount to avoid transactions running out of gas and failing if the amount of iterations is too high.

while

If you don't know how many times you want to execute the code but want to break the loop based on a condition, you can use a `while` loop (line 20). Loops are seldom used in Solidity since transactions might run out of gas and fail if there is no limit to the number of iterations that can occur.

do while

The `do while` loop is a special kind of while loop where you can ensure the code is executed at least once, before checking on the condition.

Compiled

differsAndConstructors.sol

inputsAndOutputs.sol

visibility.sol

ifElse.sol

loops.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Loop {
5     uint public count;
6     function loop() public {
7         // for loop
8         for (uint i = 0; i < 10; i++) {
9             if (i == 5) {
10                 // Skip to next iteration with continue
11                 continue;
12             }
13             if (i == 5) {
14                 // Exit loop with break
15                 break;
16             }
17             count++;
18         }
19
20         // while loop
21         uint j;
22         while (j < 10) {
23             j++;
24         }
25     }
26 }
27
```

Explain contract

8.1 arrays.sol

LEARNETH

Tutorials list

Syllabus

8.1 Data Structures - Arrays

12 / 19

8.1 Data Structures - Arrays

In the next sections, we will look into the data structures that we can use to organize and store our data in Solidity.

Arrays, mappings and structs are all *reference types*. Unlike *value types* (e.g. *booleans* or *integers*) reference types don't store their value directly. Instead, they store the location where the value is being stored. Multiple reference type variables could reference the same location, and a change in one variable would affect the others, therefore they need to be handled carefully.

In Solidity, an array stores an ordered list of values of the same type that are indexed numerically.

There are two types of arrays, compile-time *fixed-size* and *dynamic arrays*. For fixed-size arrays, we need to declare the size of the array before it is compiled. The size of dynamic arrays can be changed after the contract has been compiled.

Declaring arrays

We declare a fixed-size array by providing its type, array size (as an integer in square brackets), visibility, and name (line 9).

We declare a dynamic array in the same manner. However, we don't provide an array size and leave the brackets empty (line 6).

Compiled

ctors.sol

inputsAndOutputs.sol

visibility.sol

ifElse.sol

loops.sol

arrays.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Array {
5     // Several ways to initialize an array
6     uint[] public arr;
7     uint[] public arr2 = [1, 2, 3];
8     // Fixed sized arrays, all elements initialize to 0
9     uint[10] public myFixedSizeArr;
10    uint[3] public arr3 = [0, 1, 2];
11
12    function get(uint i) public view returns (uint) {
13        return arr[i];
14    }
15
16    // Solidity can return the entire array.
17    // But this function should be avoided for
18    // arrays that can grow indefinitely in length.
19    function getArr() public view returns (uint[3] memory) {
20        return arr3;
21    }
22
23    function push(uint i) public {
24        // Append to array
25        // This will increase the array length by 1.
26        arr.push(i);
27    }
28
29    function pop() public {
30        // Remove last element from array
31        // This will decrease the array length by 1
32        arr.pop();
33    }
34 }
```


8.2 mapping.sol

LEARNETH

Tutorials list

Syllabus

8.2 Data Structures - Mappings

13 / 19

8.2 Data Structures - Mappings

In Solidity, *mappings* are a collection of key types and corresponding value type pairs.

The biggest difference between a mapping and an array is that you can't iterate over mappings. If we don't know a key we won't be able to access its value. If we need to know all of our data or iterate over it, we should use an array.

If we want to retrieve a value based on a known key we can use a mapping (e.g. addresses are often used as keys). Looking up values with a mapping is easier and cheaper than iterating over arrays. If arrays become too large, the gas cost of iterating over it could become too high and cause the transaction to fail.

We could also store the keys of a mapping in an array that we can iterate over.

Creating mappings

Mappings are declared with the syntax `mapping(KeyType => ValueType) VariableName`. The key type can be any built-in value type or any contract, but not a reference type. The value type can be of any type.

In this contract, we are creating the public mapping `balances` (line 6) that associates the key type `address` with the value type `uint`.

Accessing values

Compiled

visibility.sol ifElse.sol loops.sol arrays.sol mappings.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Mapping {
5     // Mapping from address to uint
6     mapping(address => uint) public balances;
7
8     function get(address _addr) public view returns (uint) {
9         // Mapping always returns a value.
10        // If the value was never set, it will return the default value.
11        return balances[_addr];
12    }
13
14    function set(address _addr) public {
15        // Update the value at this address
16        balances[_addr] = _addr.balance;
17    }
18
19    function remove(address _addr) public {
20        // Reset the value to the default value.
21        delete balances[_addr];
22    }
23 }
24
25 contract NestedMapping {
26     // Nested mapping (mapping from address to another mapping)
27     mapping(address => mapping(uint => bool)) public nested;
28
29     function get(address _addr1, uint _i) public view returns (bool) {
30        // You can get values from a nested mapping
31        // even when it is not initialized
32        return nested[_addr1][_i];
33    }
34 }
```

8.3 structs.sol

LEARNETH

Tutorials list

Syllabus

8.3 Data Structures - Structs

14 / 19

8.3 Data Structures - Structs

In Solidity, we can define custom data types in the form of *structs*. Structs are a collection of variables that can consist of different data types.

Defining structs

We define a struct using the `struct` keyword and a name (line 5). Inside curly braces, we can define our struct's members, which consist of the variable names and their data types.

Initializing structs

There are different ways to initialize a struct.

Positional parameters: We can provide the name of the struct and the values of its members as parameters in parentheses (line 16).

Key-value mapping: We provide the name of the struct and the keys and values as a mapping inside curly braces (line 19).

Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Compiled

visibility.sol ifElse.sol loops.sol arrays.sol mappings.sol structs.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Todos {
5     struct Todo {
6         string text;
7         bool completed;
8     }
9
10    // An array of 'Todo' structs
11    Todo[] public todos;
12
13    function create(string memory _text) public {
14        // 3 ways to initialize a struct
15        // - calling it like a function
16        todos.push(Todo(_text, false));
17
18        // key value mapping
19        todos.push(Todo({text: _text, completed: false}));
20
21        // initialize an empty struct and then update it
22        Todo memory todo;
23        todo.text = _text;
24        // todo.completed initialized to false
25
26        todos.push(todo);
27    }
28
29    // Solidity automatically created a getter for 'todos' so
30    // you don't actually need this function.
31    function get(uint _index) public view returns (string memory text, bool completed) {
32        Todo storage todo = todos[_index];
33    }
34 }
```

8.4 enums.sol

LEARNETH

Tutorials list

Syllabus

8.4 Data Structures - Enums

15 / 19

8.4 Data Structures - Enums

In Solidity *enums* are custom data types consisting of a limited set of constant values. We use enums when our variables should only get assigned a value from a predefined set of values.

In this contract, the state variable `status` can get assigned a value from the limited set of provided values of the enum `Status` representing the various states of a shipping status.

Defining enums

We define an enum with the `enum` keyword, followed by the name of the custom type we want to create (line 6). Inside the curly braces, we define all available members of the enum.

Initializing an enum variable

We can initialize a new variable of an enum type by providing the name of the enum, the visibility, and the name of the variable (line 16). Upon its initialization, the variable will be assigned the value of the first member of the enum, in this case, `Pending` (line 7).

Even though enum members are named when you define them, they are stored as unsigned integers, not strings. They are numbered in the order that they were defined, the first member starting at 0. The initial value of `status`, in this case, is 0.

Compiled

enums.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Enum {
5     // Enum representing shipping status
6     enum Status {
7         Pending,
8         Shipped,
9         Accepted,
10        Rejected,
11        Canceled
12    }
13
14    enum Size {
15        S,
16        M,
17        L
18    }
19
20    // Default value is the first element listed in
21    // definition of the type, in this case "Pending"
22    Status public status;
23    Size public sizes;
24
25    function get() public view returns (Status) { 2605 gas
26        return status;
27    }
28
29    function getSize() public view returns (Size) { 2633 gas
30        return sizes;
31    }
32}
```

9. dataLocations.sol

LEARNETH

Tutorials list

Syllabus

9. Data Locations

16 / 19

9. Data Locations

The values of variables in Solidity can be stored in different data locations: *memory*, *storage*, and *calldata*.

As we have discussed before, variables of the value type store an independent copy of a value, while variables of the reference type (array, struct, mapping) only store the location (reference) of the value.

If we use a reference type in a function, we have to specify in which data location their values are stored. The price for the execution of the function is influenced by the data location; creating copies from reference types costs gas.

Storage

Values stored in *storage* are stored permanently on the blockchain and, therefore, are expensive to use.

In this contract, the state variables `arr`, `map`, and `myStructs` (lines 5, 6, and 10) are stored in storage. State variables are always stored in storage.

Memory

Values stored in *memory* are only stored temporarily and are not on the blockchain. They only exist during the execution of an external function and are discarded afterward. They are cheaper to use than values stored in *storage*.

Compiled

dataLocations.sol 2

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract DataLocations {
5     uint[] public arr;
6     mapping(uint => address) map;
7     struct MyStruct {
8         uint foo;
9     }
10    mapping(uint => MyStruct) public myStructs;
11
12    function f() public returns (MyStruct memory, MyStruct memory, MyStruct memory) { infinite gas
13        // call _f with state variables
14        _f(arr, map, myStructs[1]);
15        // get a struct from a mapping
16        MyStruct storage myStruct = myStructs[1];
17        myStruct.foo = 4;
18        // create a struct in memory
19        MyStruct memory myMemStruct = MyStruct(0);
20        MyStruct memory myMemStruct2 = myMemStruct;
21        myMemStruct2.foo = 1;
22
23        MyStruct memory myMemStruct3 = myStruct;
24        myMemStruct3.foo = 3;
25        return (myStruct, myMemStruct2, myMemStruct3);
26    }
27
28    function _f(
29        uint[] storage _arr,
30        mapping(uint => address) storage _map,
31        MyStruct storage _myStruct
32    ) internal {
```

10.1 etherAndWei.sol

LEARNETH

Tutorials list

Syllabus

10.1 Transactions - Ether and Wei

17 / 19

number.

wei

Wei is the smallest subunit of *Ether*, named after the cryptographer *Wei Dai*. *Ether* numbers without a suffix are treated as `wei` (line 7).

gwei

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

ether

One `ether` is equal to 1,000,000,000,000,000 (10^{18}) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

★ Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

oops.sol

arrays.sol

mappings.sol

structs.sol

enums.sol

etherAndWei.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12
13    uint public oneGwei = 1 gwei;
14    // 1 ether is equal to 10^9 wei
15    bool public isOneGwei = 1 gwei == 1e9;
16 }
```

10.2 gasAndGasPrice.sol

LEARNETH

Tutorials list

Syllabus

10.2 Transactions - Gas and Gas Price

18 / 19

Gas prices are denoted in gwei.

Gas limit

When sending a transaction, the sender specifies the maximum amount of gas that they are willing to pay for. If they set the limit too low, their transaction can run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about gas on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

★ Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

Check Answer

Show answer

Next

Well done! No errors.

Compile

ps.sol

arrays.sol

mappings.sol

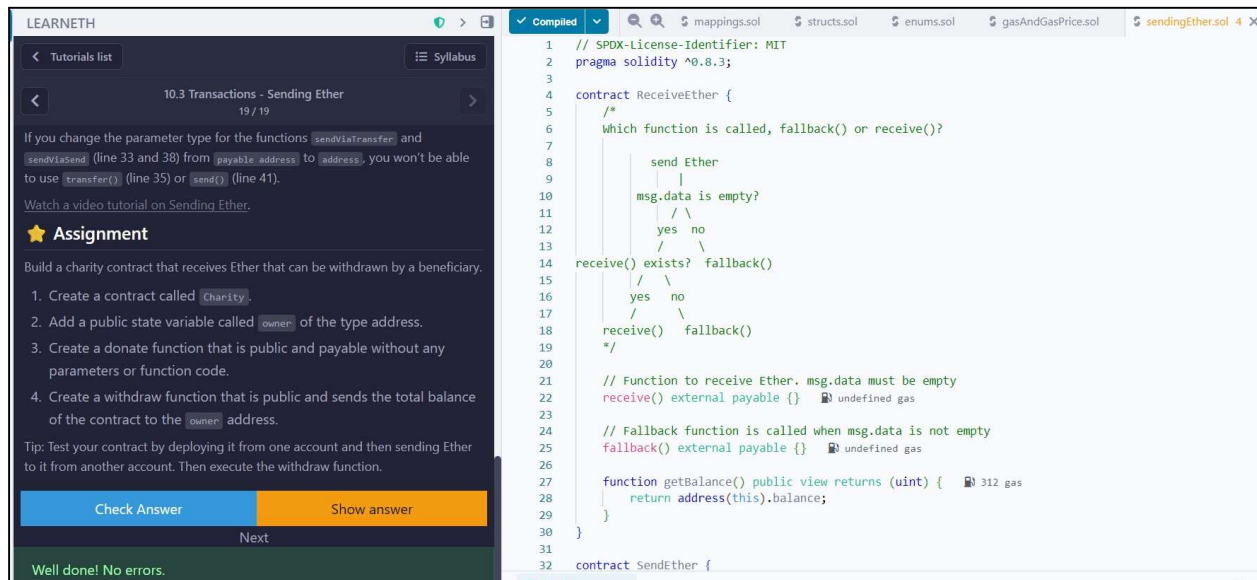
structs.sol

enums.sol

gasAndGasPrice.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public ( gas infinite gas
12        // Here we run a loop until all of the gas are spent
13        // and the transaction falls
14        while (true) {
15            i += 1;
16        }
17    }
18 }
```

10.3 sendingEther.sol



The screenshot displays the LEARNETH IDE interface. On the left, a sidebar shows the 'Tutorials list' with the current tutorial '10.3 Transactions - Sending Ether' selected. Below the title, there is a note about parameter types for `sendViaTransfer` and `sendViaSend`, a video tutorial link, and an 'Assignment' section with four steps: 1. Create a contract called `Charity`. 2. Add a public state variable called `owner` of the type `address`. 3. Create a donate function that is public and payable without any parameters or function code. 4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address. A tip at the bottom suggests testing the contract by deploying it and sending Ether. At the bottom of the sidebar are buttons for 'Check Answer', 'Show answer', and 'Next', along with a green status bar indicating 'Well done! No errors.'

The main editor on the right shows the Solidity code for `contract ReceiveEther`. The code includes a pragma statement for Solidity version `^0.8.3`, a comment about which function is called (`fallback()` or `receive()`), and a flowchart illustrating the logic: 'send Ether' leads to 'msg.data is empty?'. If 'yes', it calls `receive()`; if 'no', it calls `fallback()`. The `receive()` function is marked as 'external payable' and has a gas cost of 'undefined gas'. The `fallback()` function is also marked as 'external payable' and has a gas cost of 'undefined gas'. A `getBalance()` function is defined as 'public view returns (uint)' with a gas cost of '312 gas'. The code ends with the definition of `contract SendEther`.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ReceiveEther {
5     /*
6      * Which function is called, fallback() or receive()?
7      *
8      * send Ether
9      *
10     * msg.data is empty?
11     * / \
12     * yes no
13     * / \
14     * receive() exists? fallback()
15     * / \
16     * yes no
17     * / \
18     * receive() fallback()
19     */
20
21     // Function to receive Ether. msg.data must be empty
22     receive() external payable {} // undefined gas
23
24     // Fallback function is called when msg.data is not empty
25     fallback() external payable {} // undefined gas
26
27     function getBalance() public view returns (uint) { // 312 gas
28         return address(this).balance;
29     }
30 }
31
32 contract SendEther {
```

Conclusion

This experiment provides practical exposure to Solidity programming by covering essential blockchain concepts such as data types, variables, functions, control structures, data storage, and transaction handling. By mastering these fundamentals, developers can design secure, efficient, and scalable smart contracts that form the backbone of decentralized applications.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab
Experiment 05

Aim: Deploying a Voting/Ballot Smart Contract

Roll No.	12
Name	Palak Chanchlani
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks.

Name- Palak Chanchlani
Class - D20A
Roll no.: 12

Experiment 5

Aim - Deploying a Voting/Ballot Smart Contract

Theory :

1. Relevance of require Statements in Solidity Programs

In Solidity, the require statement is used as a safeguard to validate conditions before executing the rest of the function. If the condition fails, the transaction is reverted and no state changes are saved. This ensures that invalid actions do not corrupt contract data.

For example, in a voting contract:

- Checking if the caller is the chairperson before granting rights.
- Ensuring a voter has not already voted.

Confirming that a voter has the right to vote before allowing them to cast a ballot.

- Thus, require statements enforce correctness, security, and prevent misuse of contract functions.

2. Keywords: mapping, storage, and memory

mapping:

- Mappings are key-value data structures in Solidity, often used to associate addresses with voter information. They are efficient for lookups but cannot be iterated directly. For example:

solidity

```
mapping(address => Voter) public voters;
```

This links each Ethereum account to its voter details.

storage:

- Storage refers to permanent contract data saved on the blockchain. Variables declared at contract level are stored here. Since writing to storage consumes gas, it is costly but persistent. Example: voter details stored in the voters mapping remain available throughout the contract lifecycle.

memory:

- Memory is temporary data storage used only during function execution. It is cheaper than storage and discarded once the function ends. For instance, proposal names passed to the constructor are stored in memory during execution.

3. Why bytes32 Instead of string?

Earlier versions of the Ballot contract used bytes32 for proposal names because it is fixed-size, cheaper in gas, and faster for comparisons. However, it restricts names to 32 characters.

On the other hand, string is dynamic and user-friendly, allowing longer names but consuming more gas.

Modern contracts often prefer string for readability, even though bytes32 remains useful when efficiency is the priority.

Code :

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {
    struct Voter {
        uint weight; // weight is accumulated by delegation
        bool voted; // if true, that person already voted
        address delegate; // person delegated to
        uint vote; // index of the voted proposal
    }

    struct Proposal {
        string name; // proposal name (string for readability)
        uint voteCount; // number of accumulated votes
    }

    address public chairperson;
    mapping(address => Voter) public voters;
    Proposal[] public proposals;

    constructor(string[] memory proposalNames) {
        chairperson = msg.sender;
        voters[chairperson].weight = 1;
        for (uint i = 0; i < proposalNames.length; i++) {
            proposals.push(Proposal({
                name: proposalNames[i],
                voteCount: 0
            }));
        }
    }
}
```

```

function giveRightToVote(address voter) public {
    require(msg.sender == chairperson, "Only chairperson can give right to vote");
    require(!voters[voter].voted, "The voter already voted");
    require(voters[voter].weight == 0, "Voter already has right");
    voters[voter].weight = 1;
}

```

```

function delegate(address to) public {
    Voter storage sender = voters[msg.sender];
    require(sender.weight != 0, "You have no right to vote");
    require(!sender.voted, "You already voted");
    require(to != msg.sender, "Self-delegation is disallowed");

    while (voters[to].delegate != address(0)) {
        to = voters[to].delegate;
        require(to != msg.sender, "Found loop in delegation");
    }

    Voter storage delegate_ = voters[to];
    require(delegate_.weight >= 1, "Delegate has no right to vote");

    sender.voted = true;
    sender.delegate = to;

    if (delegate_.voted) {
        proposals[delegate_.vote].voteCount += sender.weight;
    } else {
        delegate_.weight += sender.weight;
    }
}

```

```

function vote(uint proposal) public {
    Voter storage sender = voters[msg.sender];
    require(sender.weight != 0, "Has no right to vote");
    require(!sender.voted, "Already voted");
    sender.voted = true;
    sender.vote = proposal;
    proposals[proposal].voteCount += sender.weight;
}

```

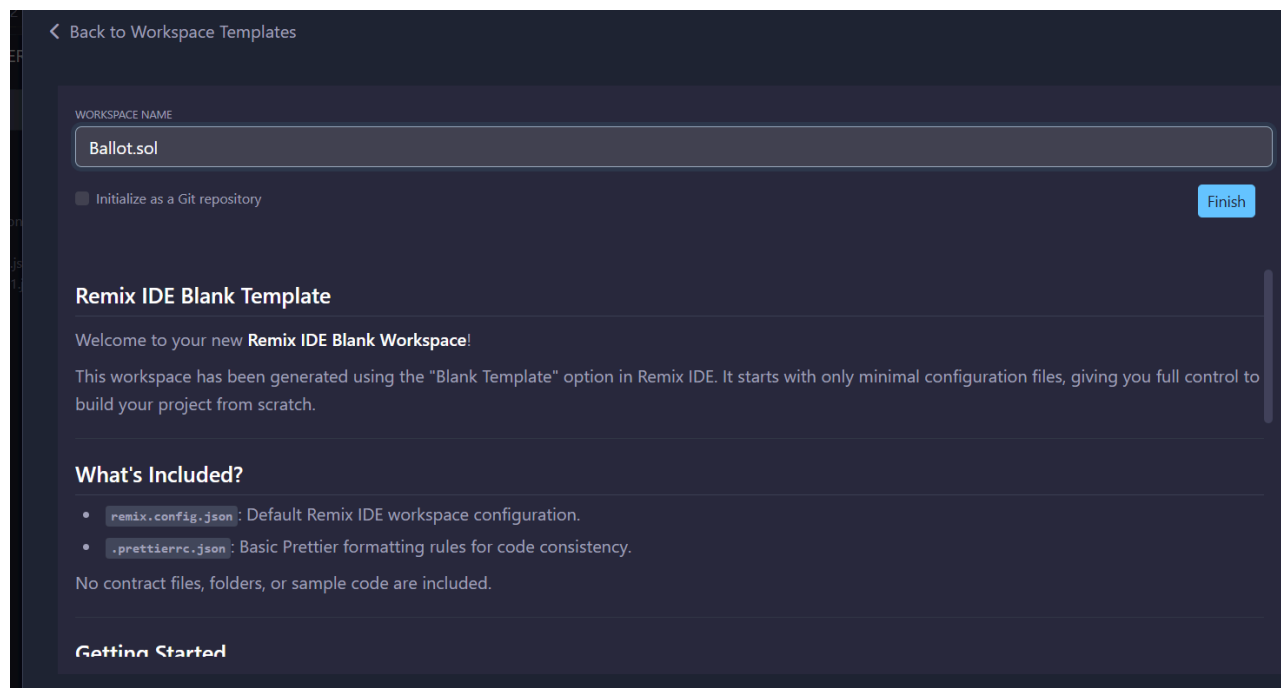
```

function winningProposal() public view returns (uint winningProposal_) {
    uint winningVoteCount = 0;
    for (uint p = 0; p < proposals.length; p++) {
        if (proposals[p].voteCount > winningVoteCount) {
            winningVoteCount = proposals[p].voteCount;
            winningProposal_ = p;
        }
    }
}

function winnerName() public view returns (string memory winnerName_) {
    winnerName_ = proposals[winningProposal()].name;
}
}

```

Output :



REMIX2.0.2

Ballot.sol

DEPLOY & RUN TRANSACTIONS

Environment

ForkReset

Remix VM

Osaka

Account 1

0x5B3...eddC4

99.999 ETH

Ballot

Ballot.sol

Compiled

proposalNames

string[]

["Palak"]

Call data

Parameters

Value

0

wei

Gas limit

auto

0

Deploy

Deployed Contracts 1

Add Contract

Compiled

Home

Ballot.sol

.prettierrc.json

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 /**
5  * @title Ballot
6  * @dev Implements voting process along with vote delegation
7  */
8 contract Ballot {
9     struct Voter {
10         uint weight; // voting weight
11         bool voted; // has the voter voted
12         address delegate; // to whom the vote is delegated
13         uint vote; // index of the voted proposal
14     }
15
16     struct Proposal {
17         string name; // proposal name (changed from bytes32 to string)
18         uint voteCount; // number of votes received
19     }
20
21     address public chairperson;
```

Explain contract

AI copilot

0

Listen on all transactions

Filter with transaction hash or ad...

• ethers.js

Type the library name to see available commands.

✓

[vm] from: 0x5B3...eddC4 to: Ballot.(constructor) value: 0 wei

data: 0x608...0000 logs: 0 hash: 0xfa5...c9692

Debug

0x5B3...eddC4

99.999 ETH

Low level interaction

non-payable

giveRightToVote

address

0xd9145CCE52D386f254917e481eB44e9943F39138

Value

0

wei

Gas limit

auto

0

Transact

Balance

0 ETH

9

```
struct Voter {
10     uint weight; // voting weight
11     bool voted; // has the voter voted
12     address delegate; // to whom the vote is delegated
13     uint vote; // index of the voted proposal
14 }
```

Explain contract

AI copilot

0

Listen on all transactions

Filter with transaction hash or ad...

• Execute JavaScript scripts:

- Input a script directly in the command line interface

- Select a Javascript file in the file explorer and then run `remix.execute()` or `remix.executeCurrent()` in the command line interface

- Right-click on a JavaScript file in the file explorer and then click `Run`

The following libraries are accessible:

• ethers.js

Type the library name to see available commands.

✓

[vm] from: 0x5B3...eddC4 to: Ballot.(constructor) value: 0 wei

data: 0x608...0000 logs: 0 hash: 0xfa5...c9692

Debug

✓

[vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xd91...39138

value: 0 wei data: 0x9e7...39138 logs: 0 hash: 0xf71...b911c

Debug

cam Alert

Initialize as git repo

Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team.

DEPLOY & RUN TRANSACTIONS

Environment

Fork

Reset

Remix VM

Osaka

Account 3

100.000 ETH

0x4B2...C02db

Deploy

Remix VM osaka

Ballot

Ballot.sol

Compiled

proposalNames

string[]

["Palak"]

Call data

Parameters

Value

0

wei

Gas limit

auto

0

Deploy

Compile

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 /**
5  * @title Ballot
```

Explain contract

0

Listen on all

[vm] from: 0x5B3...ed

value: 0 wei data: 0x9

[vm] from: 0x5B3...ed

value: 0 wei data: 0x9

transact to Ballot.giveRightToV

revert

The transaction has been

Reason provided by the contract:

If the transaction failed for not

Error occurred: revert.

revert

The transaction has been

Reason provided by the contract:

If the transaction failed for not

If the transaction failed for not

[vm] from: 0x5B3...ed

value: 0 wei data: 0x9

Scan Alerts Initialize as git repo Did you know? To protect on s

Environment

Fork

Reset

Remix VM

Osaka

Account 3

99.999 ETH

0x4B2...C02db

call proposals uint256

call voters address

call winnerName

Low level interaction

+

call winnerName

ostring: Palak

Call

Balance

0 ETH

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 /**
5  * @title Ballot
```

Explain contract

AI copilot

0

Listen on all transactions

Filter with transaction hash or ad...

transact to Ballot.giveRightToVote errored: Error occurred: Error occurred: revert.

revert

The transaction has been reverted to the initial state.

Reason provided by the contract: "Voter already has right".

If the transaction failed for not having enough gas, try increasing the gas limit gently..

Error occurred: revert.

revert

The transaction has been reverted to the initial state.

Reason provided by the contract: "Voter already has right".

If the transaction failed for not having enough gas, try increasing the gas limit gently.

If the transaction failed for not having enough gas, try increasing the gas limit gently.

[vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xd91...39138

value: 0 wei data: 0x9e7...c02db logs: 0 hash: 0x488...ddf3c

Debug

[vm] from: 0x4B2...C02db to: Ballot.vote(uint256) 0xd91...39138 value: 0 wei

data: 0x812...00000 logs: 0 hash: 0x3a0...b8e7f

Debug

[call] from: 0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db

to: Ballot.winnerName() data: 0xe2b...a53f0

Debug

Conclusion

In this experiment, a Voting/Ballot smart contract was successfully deployed and executed using Solidity on the Remix IDE. The role of require statements was observed in maintaining security and correctness by restricting unauthorized actions and preventing duplicate voting. The use of mapping helped in linking voter addresses with their details, while the distinction between storage and memory highlighted how Solidity manages permanent and temporary data efficiently. The difference between bytes32 and string for proposal names was also understood, showing the trade-off between gas optimization and user-friendly readability. Overall, the experiment demonstrated how blockchain smart contracts can enforce transparent voting rules, ensure fairness, and declare results securely on the Ethereum platform.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 06

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Roll No.	12
Name	Palak Chanchlani
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Name: Palak Chanchlani
Class: D20A
Roll No: 12

**BlockChain & DLT
EXPERIMENT NO - 6**

AIM :- Creating Smart Contracts and performing transactions using Solidity and Remix IDE

THEORY :-

1. What is a Smart Contract?

Answer:

A smart contract is a **self-executing program** that runs on the blockchain. It contains rules and conditions written in code, and once those conditions are satisfied, the contract automatically performs the specified actions. Unlike traditional agreements, smart contracts do not require intermediaries such as banks or lawyers. This makes transactions faster, cheaper, and more secure. Because they are stored on the blockchain, smart contracts are **immutable** (cannot be changed once deployed) and **transparent** (anyone can verify the rules). They are widely used in areas like finance, supply chain, voting, and digital identity management, ensuring trust and efficiency in decentralized systems.

2. How was the Smart Contract implemented in this experiment?

Answer:

In this experiment, the smart contract was implemented using **Solidity**, the programming language for Ethereum. The contract was written and deployed through **Remix IDE**, which provides a user-friendly environment for coding and testing. The contract was designed to store student details such as their ID, name, course, and marks. Functions were created to add new student records and retrieve existing ones. By deploying the contract on a blockchain environment, the student data becomes **tamper-proof** and can be accessed securely. This demonstrates how smart contracts can be applied in academic systems to maintain reliable records without relying on centralized databases.

3. What is the purpose of the addStudent function?

Answer:

The **addStudent** function is responsible for **storing student information** on the blockchain. It takes inputs such as student ID, name, course, and marks, and then saves them in the contract's data structure. Since blockchain storage is permanent, once the data is added, it cannot be altered or deleted without proper authorization. This ensures that student records remain secure and trustworthy. The function highlights how blockchain can be used to maintain academic data in a decentralized manner, reducing the risk of manipulation or loss of records.

4. What is the purpose of the getStudent function?

Answer:

The `getStudent` function is used to **retrieve student details** that have been stored in the contract. When a valid student ID is provided, the function returns the corresponding information such as name, course, and marks. This ensures that only existing records are accessed, preventing errors or invalid queries. By using this function, users can quickly verify student data directly from the blockchain. It demonstrates the **transparency and accessibility** of blockchain-based systems, where information can be securely retrieved without relying on centralized servers.

5. What is the role of IPFS in this experiment?

Answer:

IPFS (InterPlanetary File System) plays a crucial role in handling **large files** such as certificates or documents. Storing such files directly on the blockchain would be inefficient and costly because of high gas fees and limited storage capacity. Instead, IPFS stores the actual file and generates a unique hash (address) for it. This hash is then stored in the smart contract. Whenever the file needs to be accessed, the hash is used to retrieve it from IPFS. This approach makes the system more **efficient, scalable, and secure**, as the blockchain only stores lightweight references while IPFS manages the heavy data. In the context of this experiment, IPFS ensures that student certificates can be stored and verified without burdening the blockchain.

Implementation & Output :-

Step 1: Creation of Smart Contracts

Smart contracts named **AcademicRecords.sol** and **Verification.sol** were created using Solidity in Remix IDE. These contracts are used to store student academic records and verify certificate authenticity on the blockchain.

AcademicRecords.sol

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.19;
contract AcademicRecords {

    struct Student {
        string name;
        string course;
        uint marks;
        string ipfsHash;
        bool exists;
    }
}
```

```

mapping(string => Student) public students;

function addStudent(
    string memory _id,
    string memory _name,
    string memory _course,
    uint _marks,
    string memory _ipfsHash
) public {
    students[_id] = Student(_name, _course, _marks, _ipfsHash, true);
}

function getStudent(string memory _id)
    public view
    returns (string memory, string memory, uint, string memory)
{
    require(students[_id].exists, "Student not found");
    Student memory s = students[_id];
    return (s.name, s.course, s.marks, s.ipfsHash);
}
}

```

Verification.sol

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Verification {

    mapping(string => bool) public verified;

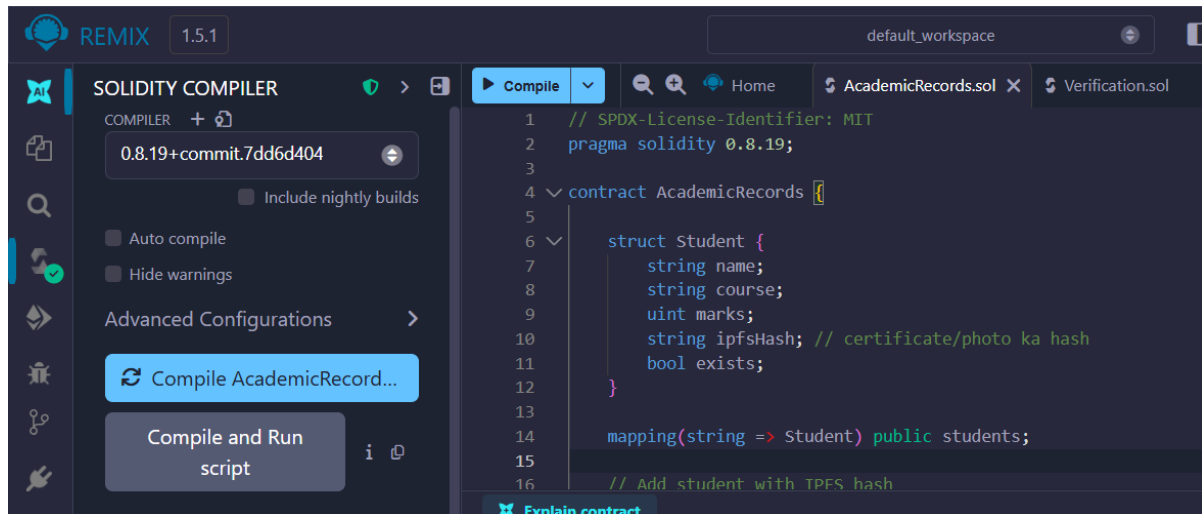
    function verifyCertificate(string memory _id) public {
        verified[_id] = true;
    }

    function checkVerification(string memory _id)
        public view
        returns (bool)
    {
        return verified[_id];
    }
}

```

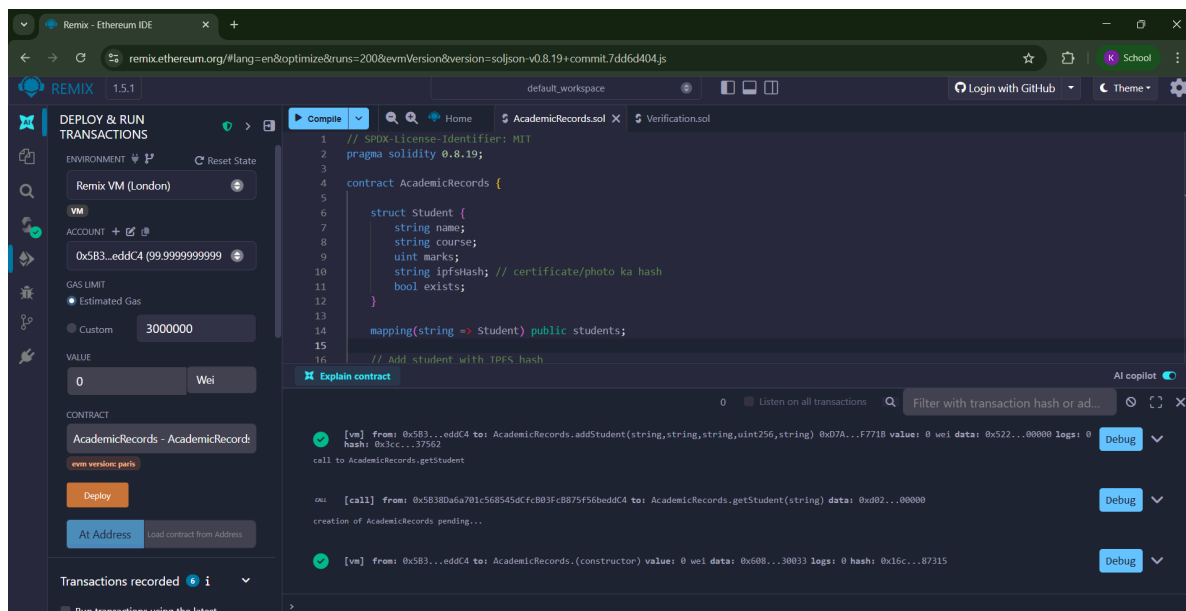
Step 2: Compilation of Smart Contracts

Both smart contracts were compiled using the Solidity Compiler in Remix IDE. The compiler version 0.8.19 was selected to ensure compatibility and successful compilation.



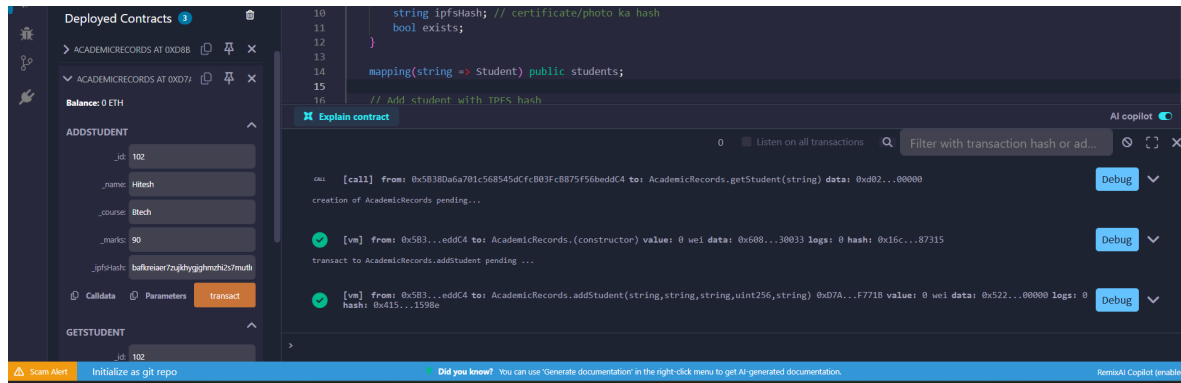
Step 3: Deployment of Smart Contracts

The contracts were deployed using the “Deploy & Run Transactions” module in Remix IDE with the JavaScript VM environment. Successful deployment generated a contract address for interaction.



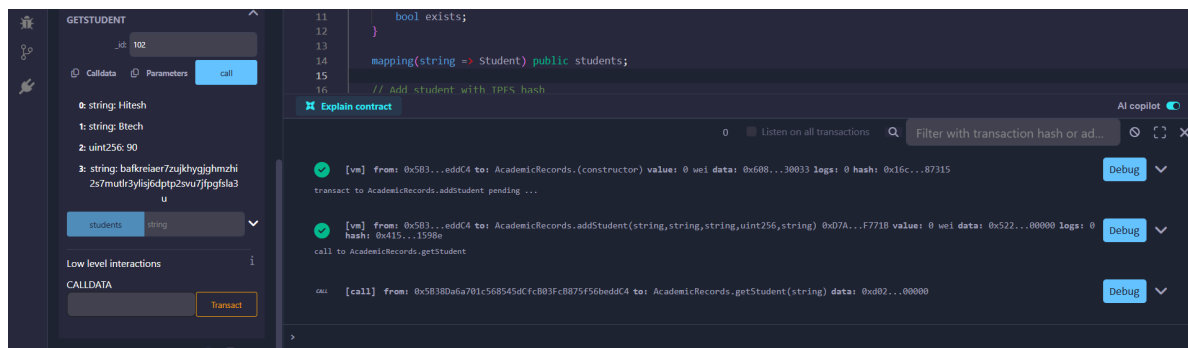
Step 4: Execution of AcademicRecords Contract

The addStudent function was executed to store student details including ID, name, course, marks, and IPFS hash. The getStudent function was then used to retrieve and verify the stored data from the blockchain.



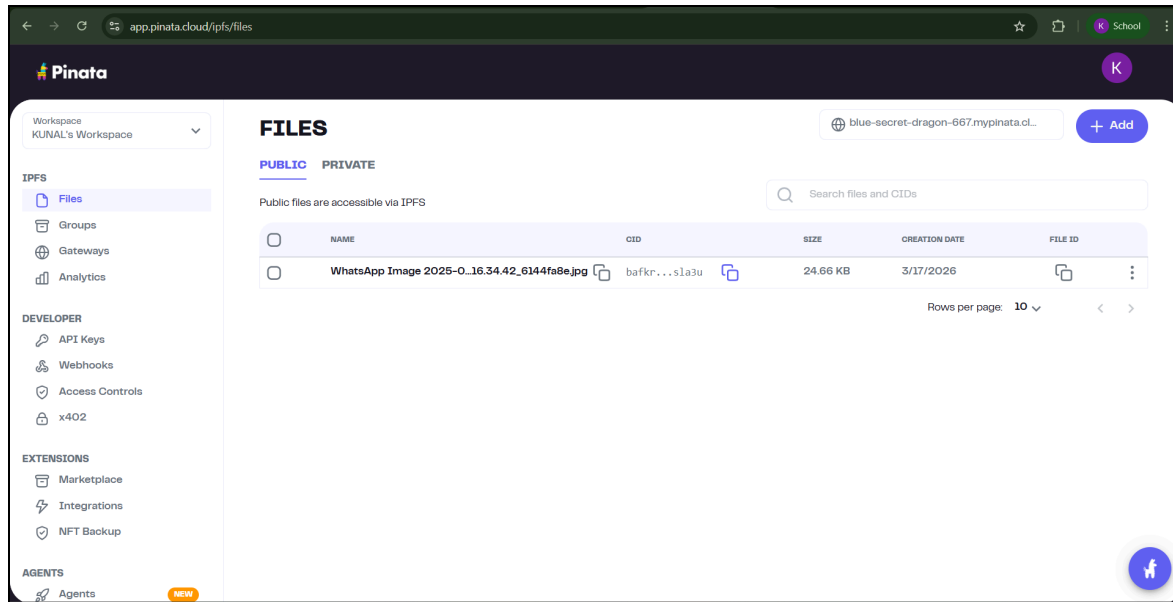
Step 5: Execution of Verification Contract

The verifyCertificate function was used to mark a certificate as verified. The checkVerification function was used to confirm whether a particular certificate is verified or not.



Step 6: Integration with IPFS

Student certificate images were uploaded to IPFS using a pinning service. The generated IPFS hash was stored in the smart contract, enabling secure storage and retrieval of documents without storing large files directly on the blockchain.



Conclusion :-

This experiment helped in understanding the practical implementation of smart contracts using Solidity in Remix IDE. A smart contract was successfully designed to store and retrieve student academic records on the blockchain, ensuring data security and immutability. The system was further enhanced by integrating IPFS to store certificate images, with only the hash stored on the blockchain. This approach makes the system efficient, secure, and resistant to data tampering. Overall, the experiment demonstrated how blockchain technology can be effectively used for reliable academic credential verification.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab

Experiment 07

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Roll No.	12
Name	Palak Chanchlani
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Name :Palak Chanchlani
Class: D20A
Roll No: 12

Experiment No: 7

AIM:

To implement MetaMask wallet integration and perform transactions using Solidity for **Academic Credentials Verification System**

Task to be Performed:

- Install MetaMask
- Create or Import Account
- Connect to Sepolia Testnet
- Fund wallet using Faucet
- Connect MetaMask with Remix IDE
- Create Smart Contract for Academic Credentials
- Compile and Deploy Smart Contract
- Perform Transactions
- Verify stored data

Theory:

MetaMask:

MetaMask is a cryptocurrency wallet that allows users to interact with blockchain networks. It acts as a bridge between the browser and blockchain, enabling transactions and smart contract interaction.

Testnet:

A testnet is a blockchain network used for testing purposes. It uses fake cryptocurrency like Sepolia ETH, allowing developers to test smart contracts without real money.

Academic Credential Verification using Blockchain:

In this system, student academic details such as name, course, and grade are stored on the blockchain using a smart contract. This ensures data is secure, tamper-proof, and easily verifiable.

Advantages:

- Secure and tamper-proof records
- Easy verification
- No dependency on centralized authority
- Transparent system

Steps to Connect MetaMask with Remix IDE:

1. Install MetaMask extension
2. Create a wallet account
3. Select Sepolia Testnet
4. Get free ETH from faucet
5. Open Remix IDE
6. Select "Injected Provider - MetaMask"
7. Connect MetaMask
8. Compile smart contract
9. Deploy contract
10. Confirm transaction
11. Interact with functions

Program (Smart Contract):

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract AcademicCredentials {

    struct Credential {

        uint256 id;
        string studentName;
        string course;
        string grade;
        address issuedBy;
        uint256 timestamp;
    }
    mapping(uint256 => Credential) public credentials;
    event CredentialAdded(
        uint256 id,
        string studentName,
        string course,
        string grade,
        address issuedBy,
        uint256 timestamp
    );

    function addCredential(
        uint256 _id,
        string memory _studentName,
        string memory _course,
```

```

        string memory _grade
    ) public {
        require(credentials[_id].id == 0, "Already exists!");
        credentials[_id] = Credential(
            _id,
            _studentName,
            _course,
            _grade,
            msg.sender,
            block.timestamp
        );

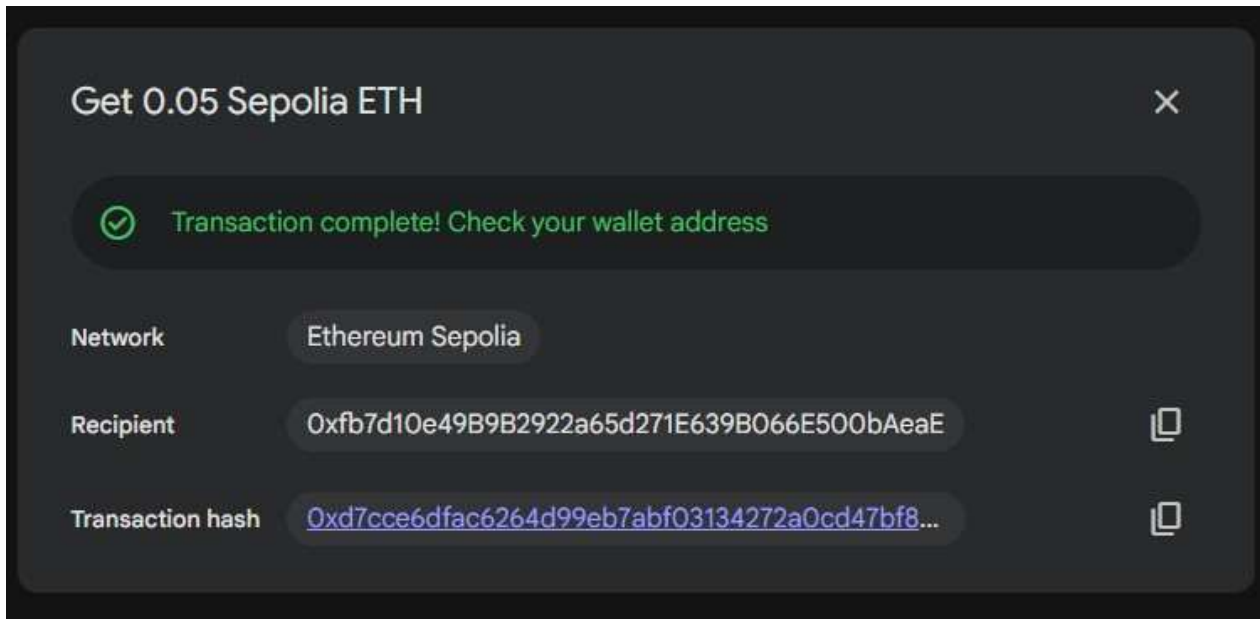
        emit CredentialAdded(
            _id,
            _studentName,
            _course,
            _grade,
            msg.sender,
            block.timestamp
        );
    }

    function getCredential(uint256 _id) public view returns (
        uint256,
        string memory,
        string memory,
        string memory,
        address,
        uint256
    ) {
        Credential memory c = credentials[_id];
        return (
            c.id,
            c.studentName,
            c.course,
            c.grade,
            c.issuedBy,
            c.timestamp
        );
    }
}

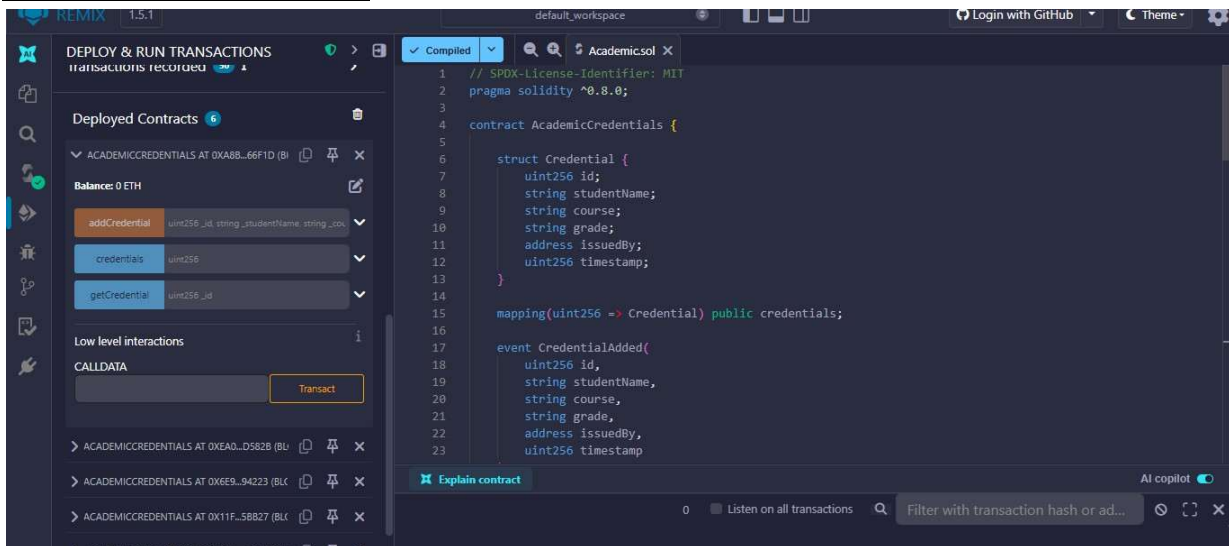
```

Output:

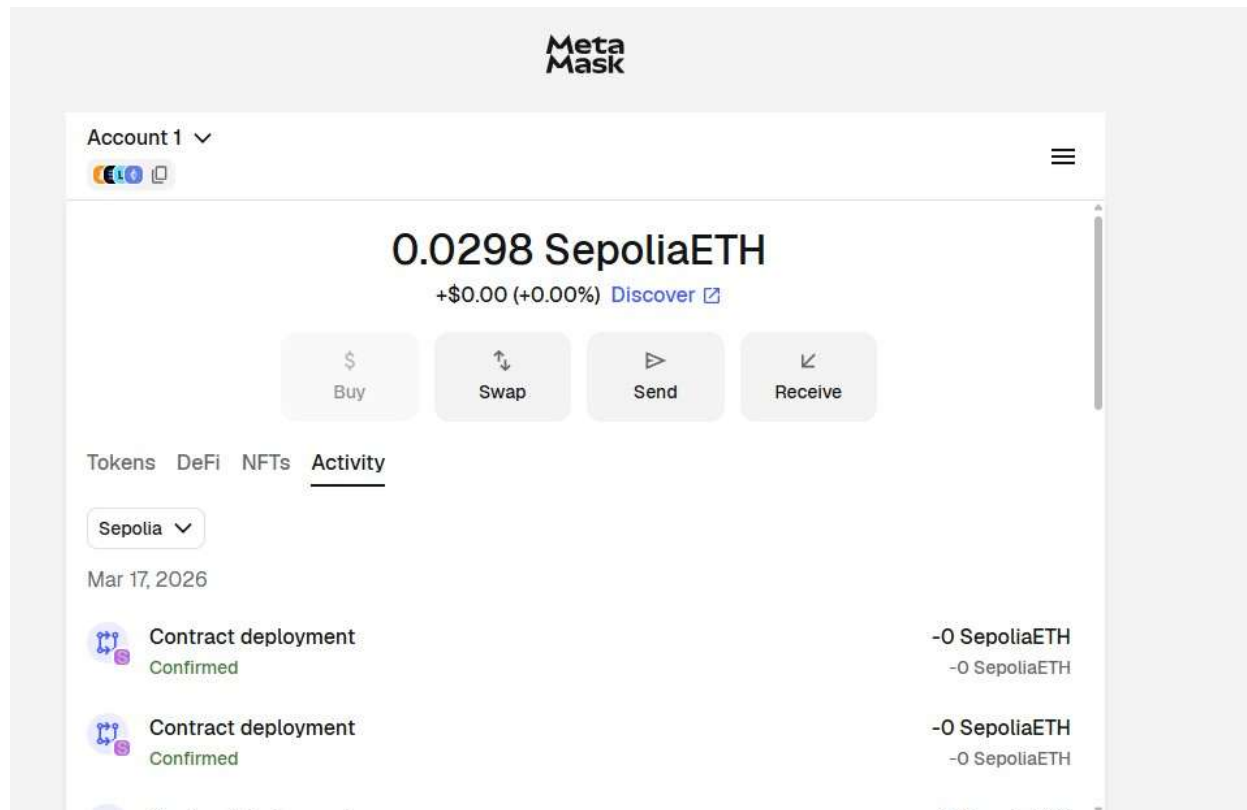
1. MetaMask Connected



2. Contract Deployment



3. Transaction Success



Conclusion:

The experiment successfully demonstrated the integration of MetaMask with Remix IDE and execution of blockchain transactions. The smart contract was deployed on Sepolia Testnet and used to store and verify academic credentials securely. This shows how blockchain can be used for tamper-proof academic verification systems.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 08

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Roll No.	12
Name	Palak Chanchlani
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Name: Palak Chanchlani
Class: D20A
Roll No: 12
BlockChain & DLT

EXPERIMENT NO - 8

Aim :- To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Theory

Q1. What is Blockchain?

Answer:

Blockchain is a distributed and decentralized digital ledger that records transactions across a network of computers (nodes). Each transaction is stored in a block, and blocks are linked together in chronological order, forming a chain. Once data is added, it cannot be altered, which ensures immutability. Because the ledger is shared among multiple participants, it provides transparency and eliminates the need for a central authority. This makes blockchain highly suitable for applications such as financial systems, supply chain management, healthcare records, and voting, where trust, security, and data integrity are essential.

Q2. What is Ganache?

Answer:

Ganache is a personal blockchain environment designed specifically for Ethereum developers. It allows developers to create a local blockchain on their computer, complete with pre-configured accounts and test Ether. This makes it possible to deploy, test, and debug smart contracts without spending real cryptocurrency. Ganache also provides features like transaction history, block details, and gas usage statistics, which help developers understand how their contracts behave in a real blockchain setting. By simulating the Ethereum network locally, Ganache speeds up the development cycle and reduces the risks of deploying untested contracts on the main network.

Q3. What is the role of MetaMask in this experiment?

Answer:

MetaMask is a digital wallet and browser extension that connects users to blockchain networks. In this experiment, MetaMask acts as a bridge between the local Ganache blockchain and the Remix IDE. It allows users to:

- Manage multiple Ethereum accounts.
- Sign and send transactions securely.
- Interact with deployed smart contracts directly from the browser.

By integrating with Ganache, MetaMask enables developers to test contracts in a realistic environment while still using familiar wallet functionalities. It ensures that transactions are properly authorized and provides a user-friendly interface for blockchain interaction.

Q4. How is Blockchain used in charity donation systems?

Answer:

Blockchain technology can transform charity donation systems by making them transparent, traceable, and trustworthy. Every donation is recorded on the blockchain, creating a permanent and verifiable record. Donors can track how their funds are being used, ensuring that money reaches the intended recipients. This reduces the risk of fraud, mismanagement, or misuse of funds. Additionally, blockchain allows for real-time reporting, meaning charities can show donors exactly where contributions are going. This level of accountability builds trust between organizations and donors, encouraging more people to contribute.

Q5. What is the role of Smart Contracts in charity donations?

Answer:

Smart contracts play a crucial role in automating the donation process. They are self-executing programs stored on the blockchain that run when predefined conditions are met. For example, a smart contract can be programmed to release funds only when a charity provides proof of completing a project. This ensures that donations are used exactly as intended. Smart contracts eliminate the need for intermediaries, reduce administrative costs, and guarantee that transactions are secure and transparent. By enforcing rules automatically, they make charity systems more efficient and reliable.

Q6. Why is Blockchain beneficial for charity systems?

Answer:

Blockchain offers several benefits for charity systems:

- **Transparency:** Donors can verify every transaction and see how funds are allocated.
- **Security:** Data stored on the blockchain cannot be tampered with, reducing fraud.
- **Accountability:** Charities are held responsible for how they use donations.
- **Cost reduction:** By removing intermediaries, administrative expenses are minimized.
- **Global reach:** Blockchain enables cross-border donations without relying on banks or third parties.

Together, these advantages make blockchain a powerful tool for building trust and efficiency in charitable organizations.

Q7. What are the steps involved in connecting Ganache with MetaMask and Remix IDE?

Answer:

The process of connecting Ganache with MetaMask and Remix IDE involves several steps:

1. Install Ganache on your computer to create a local blockchain environment.
2. Open Ganache and note down the RPC server details (usually <http://127.0.0.1:7545>).
3. Import Ganache accounts into MetaMask using the private keys provided by Ganache. This gives MetaMask access to test Ether.
4. Configure MetaMask to connect to the local Ganache network by adding a custom RPC network.
5. Open Remix IDE in the browser and select MetaMask as the environment for deploying contracts.
6. Deploy and interact with smart contracts through Remix, with MetaMask handling transaction signing and Ganache simulating the blockchain.

This setup allows developers to test contracts in a safe, controlled environment before deploying them to a live blockchain.

Implementation :-

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
```

```
contract CharityDonation {

    address public owner;
    uint public totalDonations;

    struct Donation {
        address donor;
        uint amount;
        uint timestamp;
    }

    Donation[] public donations;

    event DonationReceived(address donor, uint amount);
    event FundsWithdrawn(address owner, uint amount);

    constructor() {
        owner = msg.sender;
    }
}
```

```

// Function to donate Ether
function donate() public payable {
    require(msg.value > 0, "Donation must be greater than zero");

    donations.push(Donation(msg.sender, msg.value, block.timestamp));
    totalDonations += msg.value;

    emit DonationReceived(msg.sender, msg.value);
}

// Function to check contract balance
function getBalance() public view returns (uint) {
    return address(this).balance;
}

// Only owner can withdraw funds
function withdraw(uint amount) public {
    require(msg.sender == owner, "Only owner can withdraw funds");
    require(amount <= address(this).balance, "Insufficient balance");

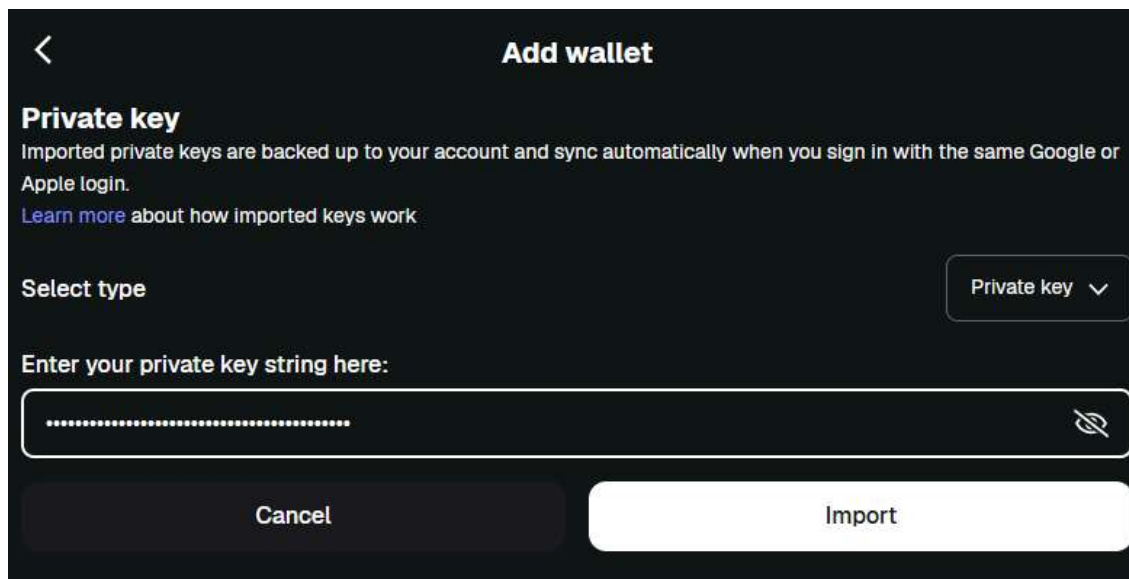
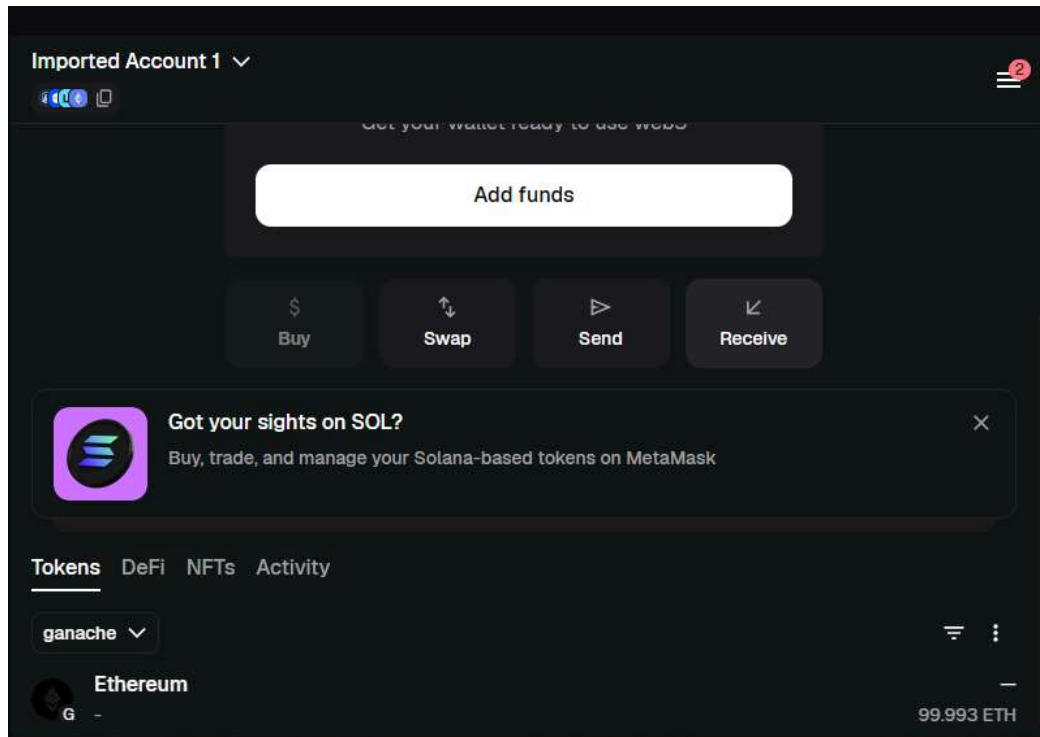
    payable(owner).transfer(amount);

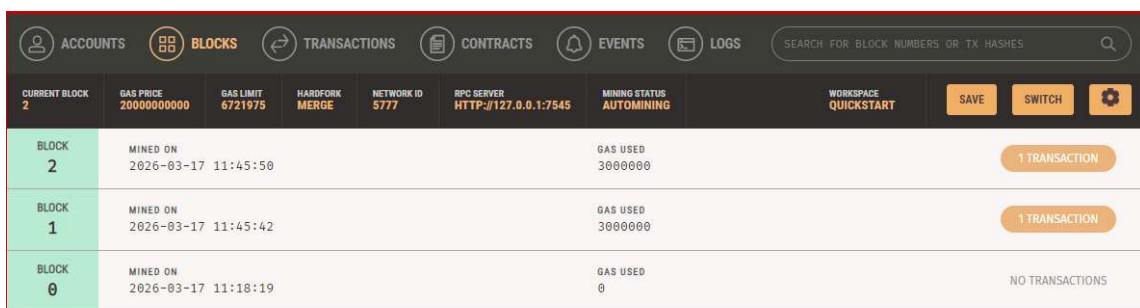
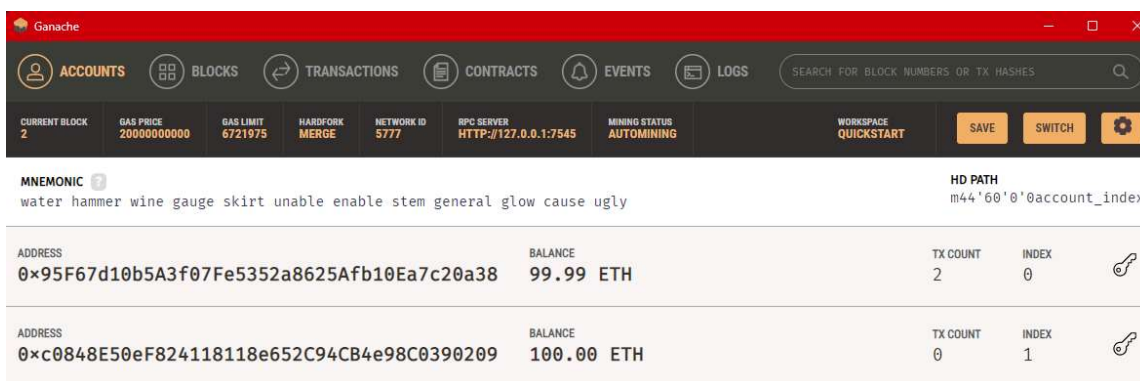
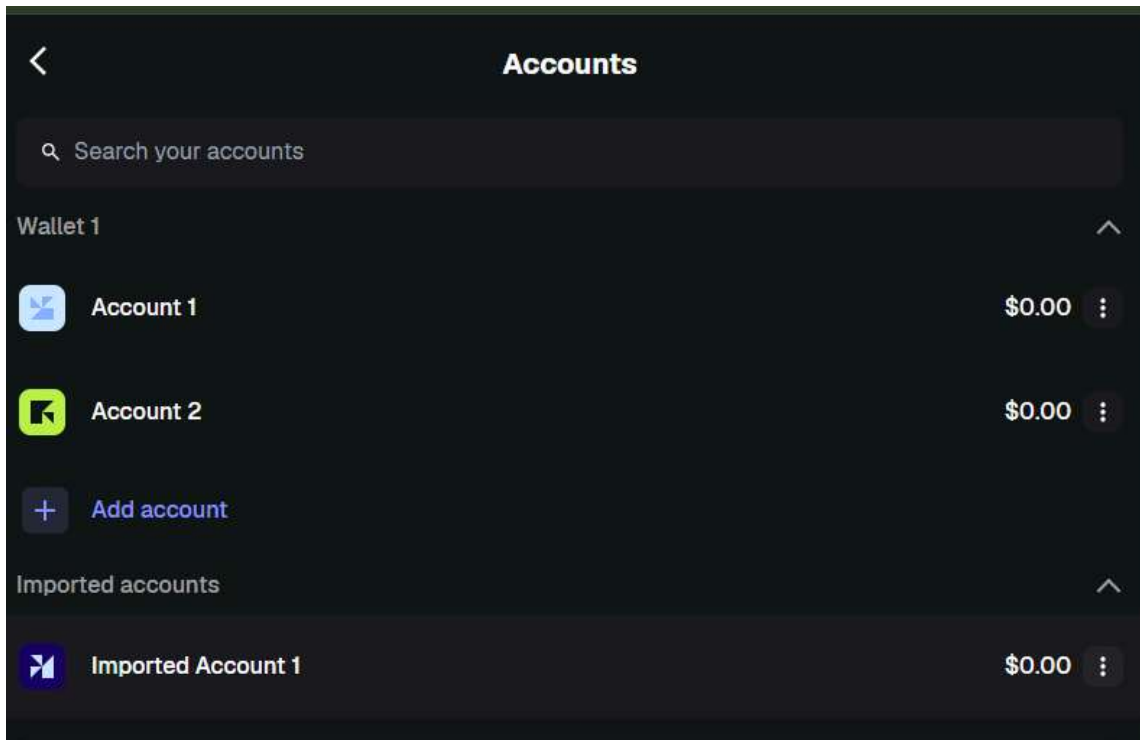
    emit FundsWithdrawn(owner, amount);
}

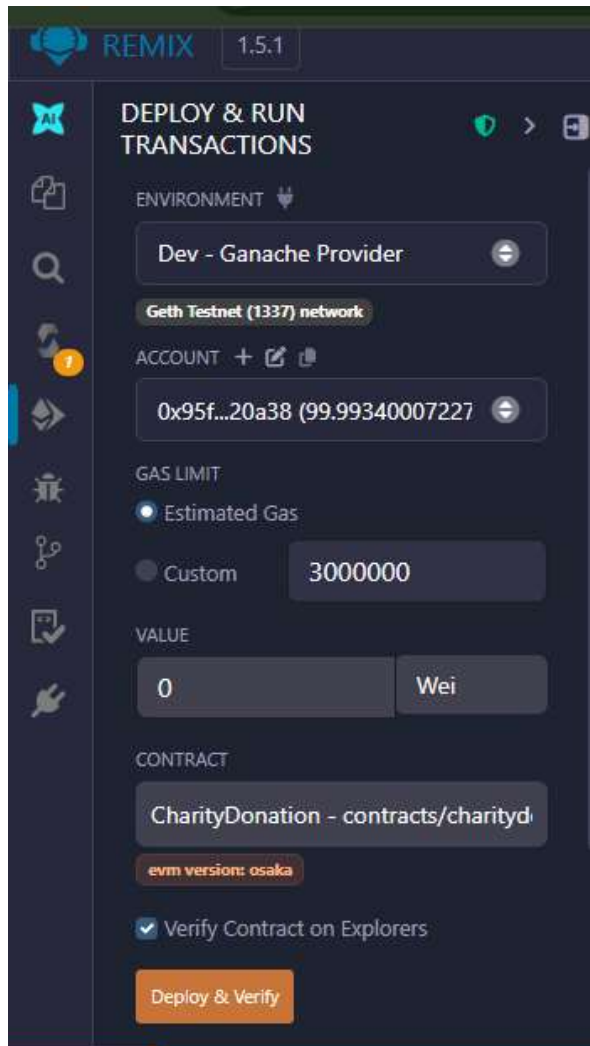
// View number of donations
function getDonationCount() public view returns (uint) {
    return donations.length;
}
}
Metamask

```

Output:







Conclusion :-

This experiment demonstrated the implementation of a blockchain-based environment using Ganache, MetaMask, and Remix IDE. A smart contract was successfully created, deployed, and tested using Ganache accounts integrated with MetaMask. The experiment also highlighted the potential of blockchain technology in enhancing charity donation systems by ensuring transparency, security, and traceability of transactions. Overall, it provided practical knowledge of executing and monitoring blockchain transactions in a controlled environment.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 09

Aim: To implement a Private Ethereum Blockchain using Geth.

Roll No.	12
Name	Palak Chanchlani
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Name: Palak Chanchlani

Class: D20A

Roll No: 12

EXPERIMENT NO.9

Aim: To implement a Private Ethereum Blockchain using Geth.

THEORY:

Private Ethereum Blockchain

Ethereum is an open-source, decentralized blockchain platform that allows developers to design and deploy smart contracts and decentralized applications (DApps). While the public Ethereum mainnet is open to anyone, a Private Ethereum Blockchain restricts participation to selected members. This makes it suitable for enterprise solutions, testing environments, and research scenarios where confidentiality, control, and performance are prioritized.

A private network provides developers with a sandbox environment to experiment with consensus mechanisms, transaction flows, and smart contract execution without incurring gas fees or relying on public validators.

Geth (Go-Ethereum)

Geth is one of the most popular Ethereum clients, developed in the Go programming language. It enables users to run Ethereum nodes, mine Ether, and interact with the blockchain through JSON-RPC, command-line tools, or JavaScript consoles. Geth is widely used for setting up private Ethereum networks because of its flexibility and ease of configuration.

Steps to Set Up a Private Ethereum Network

- **Choosing a Network ID:** Each private blockchain must have a unique network ID to distinguish it from public or other private networks.
- **Selecting a Consensus Algorithm:** Ethereum supports multiple consensus mechanisms such as Proof of Work (PoW) and Proof of Authority (PoA). In private networks, Clique (PoA) is often preferred for faster block confirmation.
- **Creating a Genesis Block:** The genesis block is the first block of the blockchain. It defines parameters such as difficulty, gas limit, initial account balances, and consensus rules.
- **Initializing the Geth Database:** Using the `geth init` command, the blockchain is initialized with the `genesis.json` file, preparing the data directory for node operation.

- Setting up Networking: Nodes communicate using enodes (Ethereum Node URLs), which contain IP addresses and public keys for peer discovery and connection.
- Running Member Nodes: Each participant runs a Geth node that connects to the private network, validates blocks, and processes transactions.
- Running a Signer (Clique Consensus): In Clique, designated signers validate and sign blocks. These accounts are pre-configured in the genesis file to ensure trusted block production.

Step 1: Installing Geth on Ubuntu

sudo apt update

```
ubuntu@ubuntu:~/Desktop$ sudo apt update
Hit:1 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease [119 kB]
Get:3 http://security.ubuntu.com/ubuntu jammy-security InRelease [110 kB]
Get:4 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu jammy InRelease [17.5 kB]
Hit:5 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease
Get:6 http://in.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [1,519 kB]
Get:7 http://security.ubuntu.com/ubuntu jammy-security/main i386 Packages [436 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [1,303 kB]
Get:9 http://security.ubuntu.com/ubuntu jammy-security/main Translation-en [233 kB]
Get:10 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [1,616 kB]
Get:11 http://security.ubuntu.com/ubuntu jammy-security/restricted Translation-en [271 kB]
Get:12 http://security.ubuntu.com/ubuntu jammy-security/universe i386 Packages [599 kB]
Get:13 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [852 kB]
Get:14 http://security.ubuntu.com/ubuntu jammy-security/universe Translation-en [163 kB]
Get:15 http://in.archive.ubuntu.com/ubuntu jammy-updates/main i386 Packages [602 kB]
Get:16 http://in.archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [293 kB]
Get:17 http://in.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [1,644 kB]
Get:18 http://in.archive.ubuntu.com/ubuntu jammy-updates/restricted Translation-en [274 kB]
Get:19 http://in.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1,060 kB]
Get:20 http://in.archive.ubuntu.com/ubuntu jammy-updates/universe i386 Packages [698 kB]
Get:21 http://in.archive.ubuntu.com/ubuntu jammy-updates/universe Translation-en [241 kB]
```

sudo apt install software-properties-common

```
ubuntu@ubuntu:~/Desktop$ sudo apt install software-properties-common
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  python3-software-properties software-properties-gtk ubuntu-advantage-tools
  ubuntu-pro-client
Recommended packages:
  ubuntu-pro-client-l10n
The following NEW packages will be installed:
  ubuntu-pro-client
The following packages will be upgraded:
  python3-software-properties software-properties-common software-properties-gtk
  ubuntu-advantage-tools
4 upgraded, 1 newly installed, 0 to remove and 231 not upgraded.
Need to get 322 kB of archives.
After this operation, 1,339 kB disk space will be freed.
Do you want to continue? [Y/n] y
Get:1 http://in.archive.ubuntu.com/ubuntu jammy-updates/main amd64 ubuntu-advantage-tools a
11 31 2-22 04 [10.8 kB]
```

sudo add-apt-repository -y ppa:ethereum/ethereum

```
ubuntu@ubuntu:~/Desktop$ sudo add-apt-repository -y ppa:ethereum/ethereum
Repository: 'deb https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu/ jammy main'
More info: https://launchpad.net/~ethereum/+archive/ubuntu/ethereum
Adding repository.
Found existing deb entry in /etc/apt/sources.list.d/ethereum-ubuntu-ethereum-jammy.list
Adding deb entry to /etc/apt/sources.list.d/ethereum-ubuntu-ethereum-jammy.list
Found existing deb-src entry in /etc/apt/sources.list.d/ethereum-ubuntu-ethereum-jammy.list
Adding disabled deb-src entry to /etc/apt/sources.list.d/ethereum-ubuntu-ethereum-jammy.lis
t
Adding key to /etc/apt/trusted.gpg.d/ethereum-ubuntu-ethereum.gpg with fingerprint 2A518C81
9BE37D2C2031944D1C52189C923F6CA9
Hit:1 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:5 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu jammy InRelease
Reading package lists... Done
```

```
ubuntu@ubuntu:~/Desktop$ sudo apt update
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu jammy InRelease
Hit:5 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
231 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ubuntu:~/Desktop$
```

```
ubuntu@ubuntu:~/Desktop$ sudo apt install geth
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
geth is already the newest version (1.13.14+build29502+jammy).
geth set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 231 not upgraded.
```

Step 2: Check the version of Geth on the Terminal

geth version

```
ubuntu@ubuntu:~/Desktop$ geth version
Geth
Version: 1.13.14-stable
Git Commit: 2bd6bd01d2e8561dd7fc21b631f4a34ac16627a1
Architecture: amd64
Go Version: go1.21.6
Operating System: linux
GOPATH=
GOROOT=
ubuntu@ubuntu:~/Desktop$
```

3. Step - 3: Create a Private Ethereum Network

1. Create a folder named, private_ethereum_setup

```
ubuntu@ubuntu:~/Desktop$ mkdir private_ethereum_setup
```

2. Create 2 subfolders named node1 and node2 in the folder private_ethereum_setup

```
ubuntu@ubuntu:~/Desktop$ cd private_ethereum_setup/
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ mkdir node1 node2
```

3. Create 2 accounts in the folder corresponding to node1 and node2

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node1 account new
INFO [03-31|10:20:39.436] Maximum peer count          ETH=50 total=50
INFO [03-31|10:20:39.438] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.
comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key:  0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE
Path of the secret key file: node1/keystore/UTC--2024-03-31T04-50-51.743374765Z--9f9497c56b548
90fd2f693bac26c71e4736eb2be

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!

ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ S
```

```

ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node2 account new
INFO [03-31|10:21:31.739] Maximum peer count          ETH=50 total=50
INFO [03-31|10:21:31.741] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.
comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0xfc09dce1AB0Fe679e5cC09D88B3E99de13864433
Path of the secret key file: node2/keystore/UTC--2024-03-31T04-51-35.934981574Z--fc09dce1ab0fe679e5cc09d88b3e99de13864433

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!

ubuntu@ubuntu:~/Desktop/private_ethereum_setup$

```

4. Create a genesis.json file in the folder, private_ethereum_setup
 nano genesis.json

```

ubuntu@ubuntu: ~/Desktop/private_ethereum_setup
GNU nano 6.2                                genesis.json *

"config": {
  "chainId": 1234,
  "homesteadBlock": 0,
  "eip150Block": 0,
  "eip155Block": 0,
  "eip158Block": 0,
  "byzantiumBlock": 0,
  "constantinopleBlock": 0,
  "petersburgBlock": 0,
  "istanbulBlock": 0,
  "ethash": {}
},
"nonce": "0x0",
"timestamp": "0x5f5a0a92",
"extraData": "0x0000000000000000000000000000000000000000000000000000000000000000",
"gasLimit": "0x8000000",
"difficulty": "0x4000",
"mixhash": "0x0000000000000000000000000000000000000000000000000000000000000000",
"coinbase": "0x0000000000000000000000000000000000000000000000000000000000000000",
"alloc": {
  "0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE": {
    "balance": "1000000000000000000000000"
  },
  "0xfc09dce1AB0Fe679e5cC09D88B3E99de13864433": {
    "balance": "1000000000000000000000000"
  }
}
}

```

5. Initialize the nodes with the genesis file

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node1 init genesis.json
INFO [03-31|10:33:21.842] Maximum peer count                      ETH=50 total=50
INFO [03-31|10:33:21.843] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.
comm: no such file or directory"
INFO [03-31|10:33:21.847] Set global gas cap                      cap=50,000,000
INFO [03-31|10:33:21.850] Initializing the KZG library            backend=gokzg
INFO [03-31|10:33:21.919] Defaulting to pebble as the backing database
INFO [03-31|10:33:21.919] Allocated cache and file handles       database=/home/ubuntu/Deskt
op/private_ethereum_setup/node1/geth/chaindata cache=16.00MiB handles=16
INFO [03-31|10:33:21.942] Opened ancient database                database=/home/ubuntu/Deskt
op/private_ethereum_setup/node1/geth/chaindata/ancient/chain readonly=false
INFO [03-31|10:33:21.943] State schema set to default            scheme=hash
INFO [03-31|10:33:21.943] Writing custom genesis block
INFO [03-31|10:33:21.945] Persisted trie from memory database     nodes=4 size=480.00B time="
609.399µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [03-31|10:33:21.953] Successfully wrote genesis state        database=chaindata hash=c63
8fa..fe1846
INFO [03-31|10:33:21.953] Defaulting to pebble as the backing database
INFO [03-31|10:33:21.953] Allocated cache and file handles       database=/home/ubuntu/Deskt
op/private_ethereum_setup/node1/geth/lightchaindata cache=16.00MiB handles=16
INFO [03-31|10:33:21.962] Opened ancient database                database=/home/ubuntu/Deskt
op/private_ethereum_setup/node1/geth/lightchaindata/ancient/chain readonly=false
INFO [03-31|10:33:21.962] State schema set to default            scheme=hash
INFO [03-31|10:33:21.963] Writing custom genesis block
INFO [03-31|10:33:21.964] Persisted trie from memory database     nodes=4 size=480.00B time="
811.122µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [03-31|10:33:21.969] Successfully wrote genesis state        database=lightchaindata has
h=c638fa..fe1846
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ S
```

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node2 init genesis.json
INFO [03-31|10:33:56.477] Maximum peer count                      ETH=50 total=50
INFO [03-31|10:33:56.479] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.
comm: no such file or directory"
INFO [03-31|10:33:56.485] Set global gas cap                      cap=50,000,000
INFO [03-31|10:33:56.486] Initializing the KZG library            backend=gokzg
INFO [03-31|10:33:56.571] Defaulting to pebble as the backing database
INFO [03-31|10:33:56.571] Allocated cache and file handles       database=/home/ubuntu/Deskt
op/private_ethereum_setup/node2/geth/chaindata cache=16.00MiB handles=16
INFO [03-31|10:33:56.583] Opened ancient database                database=/home/ubuntu/Deskt
op/private_ethereum_setup/node2/geth/chaindata/ancient/chain readonly=false
INFO [03-31|10:33:56.583] State schema set to default            scheme=hash
INFO [03-31|10:33:56.583] Writing custom genesis block
INFO [03-31|10:33:56.584] Persisted trie from memory database     nodes=4 size=480.00B time="
695.266µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [03-31|10:33:56.590] Successfully wrote genesis state        database=chaindata hash=c63
8fa..fe1846
INFO [03-31|10:33:56.590] Defaulting to pebble as the backing database
INFO [03-31|10:33:56.590] Allocated cache and file handles       database=/home/ubuntu/Deskt
op/private_ethereum_setup/node2/geth/lightchaindata cache=16.00MiB handles=16
INFO [03-31|10:33:56.602] Opened ancient database                database=/home/ubuntu/Deskt
op/private_ethereum_setup/node2/geth/lightchaindata/ancient/chain readonly=false
INFO [03-31|10:33:56.602] State schema set to default            scheme=hash
INFO [03-31|10:33:56.602] Writing custom genesis block
INFO [03-31|10:33:56.603] Persisted trie from memory database     nodes=4 size=480.00B time="
549.872µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [03-31|10:33:56.609] Successfully wrote genesis state        database=lightchaindata has
h=c638fa..fe1846
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$
```

6. For configuring the boot node

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ sudo apt-get install bootnode
[sudo] password for ubuntu:
Sorry, try again.
[sudo] password for ubuntu:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
bootnode is already the newest version (1.13.14+build29502+jammy).
bootnode set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 231 not upgraded.
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$
```

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ bootnode -genkey boot.key
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ bootnode -nodekey boot.key
enode://fbbb379450b32a560c1adbc01f89cd7bf7434c64d3d61140e35f01e4dc6872e079e610606a8127a161a0ed9d7be778636b22552290f7aaabad2e8232ed823eee@127.0.0.1:0?discport=30301
Note: you're using cmd/bootnode, a developer tool.
We recommend using a regular node as bootstrap node for production deployments.
INFO [03-31|10:35:18.634] New local node record          seq=1,711,861,518,633 id=ba197519c6e384da ip=<nil> udp=0 tcp=0
```

Step - 4: Establish a Peer-Peer Connection between the nodes along with the bootnode

1. On the first Terminal, Use boot.key to run the boot node

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ bootnode -verbosity 5 -nodekey boot.key -addr 127.0.0.1:30305
enode://fbbb379450b32a560c1adbc01f89cd7bf7434c64d3d61140e35f01e4dc6872e079e610606a8127a161a0ed9d7be778636b22552290f7aaabad2e8232ed823eee@127.0.0.1:0?discport=30305
Note: you're using cmd/bootnode, a developer tool.
We recommend using a regular node as bootstrap node for production deployments.
INFO [03-31|10:40:06.489] New local node record          seq=1,711,861,806,489 id=ba197519c6e384da ip=<nil> udp=0 tcp=0
```

2. On the second Terminal, Run Node 1

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node1 --port 30306 --bootnodes enode://f7aba85ba369
923bffd3438b4c8fde6b1f02b1c23ea0aac825ed7eac38e6230e5cadcf868e73b0e28710f4c9f685ca71a86a4911461637ae9ab2bd852939b7
7f@127.0.0.1:0?discport=30305 --networkid 1234 --unlock 0x9f9497c56b54890fd2f693baC26C71e4736eb2bE --password "./
password.txt" --authrpc.port 8551 --mine --miner.etherbase 0x9f9497c56b54890fd2f693baC26C71e4736eb2bE
INFO [03-31|11:25:31.763] Maximum peer count                      ETH=50 LES=0 total=50
INFO [03-31|11:25:31.765] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.comm: no such file o
r directory"
INFO [03-31|11:25:31.769] Set global gas cap                      cap=50,000,000
INFO [03-31|11:25:31.771] Allocated trie memory caches           clean=154.00MiB dirty=256.00MiB
INFO [03-31|11:25:31.771] Using pebble as the backing database
INFO [03-31|11:25:31.771] Allocated cache and file handles       database=/home/ubuntu/Desktop/private_ethereum_
setup/node1/geth/chaindata cache=512.00MiB handles=524,288
INFO [03-31|11:25:31.785] Opened ancient database                database=/home/ubuntu/Desktop/private_ethereum_
setup/node1/geth/chaindata/ancient/chain readonly=false
INFO [03-31|11:25:31.792] Disk storage enabled for ethash caches  dir=/home/ubuntu/Desktop/private_ethereum_setup
/node1/geth/ethash count=3
INFO [03-31|11:25:31.796] Disk storage enabled for ethash DAGs    dir=/home/ubuntu/.ethash count=2
INFO [03-31|11:25:31.797] Initialising Ethereum protocol         network=1234 dbversion=8
INFO [03-31|11:25:31.797] -----
INFO [03-31|11:25:31.797] Chain ID: 1234 (unknown)
INFO [03-31|11:25:31.797] Consensus: Ethash (proof-of-work)
INFO [03-31|11:25:31.797] Pre-Merge hard forks (block based):
INFO [03-31|11:25:31.797] - Homestead: #0                       (https://github.com/ethereum/execution-specs/b
lob/master/network-upgrades/mainnet-upgrades/homestead.md)
INFO [03-31|11:25:31.797] - Tangerine Whistle (EIP 150): #0     (https://github.com/ethereum/execution-specs/b
lob/master/network-upgrades/mainnet-upgrades/tangerine-whistle.md)
INFO [03-31|11:25:31.797] - Spurious Dragon/1 (EIP 155): #0     (https://github.com/ethereum/execution-specs/b
lob/master/network-upgrades/mainnet-upgrades/spurious-dragon.md)
INFO [03-31|11:25:31.797] - Spurious Dragon/2 (EIP 158): #0     (https://github.com/ethereum/execution-specs/b
```

ubuntu@ubuntu: ~/Desktop/private_ethereum_setup	ubuntu@ubuntu: ~/Desktop/private_ethereum_setup
tl>	
TRACE[03-31 11:27:49.599] >> NEIGHBORS/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	
TRACE[03-31 11:27:50.100] << FINDNODE/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	
TRACE[03-31 11:27:50.100] >> NEIGHBORS/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	
TRACE[03-31 11:27:50.601] << FINDNODE/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	
TRACE[03-31 11:27:50.601] >> NEIGHBORS/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	
TRACE[03-31 11:27:51.102] << FINDNODE/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	
TRACE[03-31 11:27:51.102] >> NEIGHBORS/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	
TRACE[03-31 11:27:51.602] << FINDNODE/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	
TRACE[03-31 11:27:51.602] >> NEIGHBORS/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	
TRACE[03-31 11:27:51.625] << PONG/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	
TRACE[03-31 11:27:51.625] >> PING/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	
DEBUG[03-31 11:27:51.625] Revalidated node	b=16 id=1af3a1e4b66753ff checks=2
TRACE[03-31 11:27:52.103] << FINDNODE/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	
TRACE[03-31 11:27:52.104] >> NEIGHBORS/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	
TRACE[03-31 11:27:52.604] << FINDNODE/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	
TRACE[03-31 11:27:52.604] >> NEIGHBORS/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	
TRACE[03-31 11:27:53.105] << FINDNODE/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	
TRACE[03-31 11:27:53.105] >> NEIGHBORS/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
tl>	

3. On the third Terminal, Run Node 2

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node2 --port 30307 --bootnodes enode://f7aba85ba369923bffd3438b4c8fde6b1f02b1c23ea0aac825ed7eac38e6230e5cadcf868e73b0e28710f4c9f685ca71a86a4911461637ae9ab2bd852939b77f0127.0.0.1:0?discport=30305 --networkid 1234 --keystore "/node2/keystore" --unlock 0xfc09dce1AB0Fe679e5cC09D88B3E99de13864433 --password "/password.txt" console
```

```
INFO [03-31|11:56:49.157] Maximum peer count                ETH=50 LES=0 total=50
INFO [03-31|11:56:49.159] Smartcard socket not found, disabling err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [03-31|11:56:49.164] Set global gas cap                cap=50,000,000
INFO [03-31|11:56:49.167] Allocated trie memory caches      clean=154.00MiB dirty=256.00MiB
INFO [03-31|11:56:49.167] Using pebble as the backing database
INFO [03-31|11:56:49.167] Allocated cache and file handles database=/home/ubuntu/Desktop/private_ethereum_setup/node2/geth/chaindata cache=512.00MiB handles=524,288
INFO [03-31|11:56:49.192] Opened ancient database           database=/home/ubuntu/Desktop/private_ethereum_setup/node2/geth/chaindata/ancient/chain readonly=false
INFO [03-31|11:56:49.202] Disk storage enabled for ethash caches dir=/home/ubuntu/Desktop/private_ethereum_setup/node2/geth/ethash count=3
INFO [03-31|11:56:49.202] Disk storage enabled for ethash DAGs dir=/home/ubuntu/.ethash count=2
INFO [03-31|11:56:49.203] Initialising Ethereum protocol    network=1234 dbversion=8
INFO [03-31|11:56:49.204]
INFO [03-31|11:56:49.204] -----
INFO [03-31|11:56:49.204] Chain ID: 1234 (unknown)
INFO [03-31|11:56:49.205] Consensus: Ethash (proof-of-work)
INFO [03-31|11:56:49.205] Pre-Merge hard forks (block based):
INFO [03-31|11:56:49.205]   - Homestead: #0                 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/homestead.md)
INFO [03-31|11:56:49.205]   - Tangerine Whistle (EIP 150): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/tangerine-whistle.md)
```

1. Fetch network status

```
> net
{
  listening: true,
  peerCount: 1,
  version: "1234",
  getListening: function(callback),
  getPeerCount: function(callback),
  getVersion: function(callback)
}
```

2. To fetch the number of blocks mined

```
> web3.eth.blockNumber
0
> INFO [03-23|12:15:29.133] Looking for peers peercount=1
  tried=0 static=0
```

3. To check the balance of the accounts

```
percentage=72 elapsed=0m42.708s
> eth.getBalance(eth.accounts[0])
1.000322e+24
> INFO [03-31|12:09:54.727] Generating DAG in progress
```


4. To fetch the details of the latest mined block

[illegible][illegible]

```
INFO [03-31|12:13:07.447] Looking for peers
> eth.getBalance("0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE")
1.000322e+24
INFO [03-31|12:13:17.464] Looking for peers
```

7. Fetch the details of the peers in the network

```
> admin.peers
[[
  {
    caps: ["eth/66", "eth/67", "eth/68", "snap/1"],
    enode: "enode://dce739f3f2fae54667568d84488b8d055409c862e80d78d653bf086b250ddd431fa6c6342502f9ff8f99129b5ee055679fab9335a9049ec7ae1b13941bb265e@127.0.0.1:30307",
    id: "ebf13aa79411d6e216f71bee7625c7391e7a2049d3853aed09ffb5566981dc27",
    name: "Geth/v1.11.6-stable-ea9e62ca/linux-amd64/go1.20.3",
    network: {
      inbound: false,
      localAddress: "127.0.0.1:36900",
      remoteAddress: "127.0.0.1:30307",
      static: false,
      trusted: false
    },
    protocols: {
      eth: {
        version: 68
      },
      snap: {
        version: 1
      }
    }
  }
]]
```

```
> eth.sendTransaction({from:"0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE",to:"0xfc09dce1AB0Fe679e5cC09D88B3E99de13864433"})
WARN [03-31|12:16:19.599] Caller gas above allowance, capping      requested=114,569,871 cap=50,000,000
INFO [03-31|12:16:19.617] Setting new local account      address=0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE
INFO [03-31|12:16:19.619] Submitted transaction          hash=0xf2c1e10c56397e2726e94bc166405b3ff0863de4757b479a1b6812debe56f33f from=0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE nonce=0 recipient=0xfc09dce1AB0Fe679e5cC09D88B3E99de13864433 value=0
"0xf2c1e10c56397e2726e94bc166405b3ff0863de4757b479a1b6812debe56f33f"
> INFO [03-31|12:16:27.814] Looking for peers      peercount=1 tried=0 static=0
```

9. Check the balances of sender and receiver 2

```
> eth.getBalance("0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE")
1.000322e+24
```

10. To check the details of the transaction on Node 1 Terminal

```
> eth.getTransaction("0xe181e503b93f95f2e69553091fbdd8459eda0942662c68ead0c5492779cf31c7")
{
  blockHash: null,
  blockNumber: null,
  chainId: "0x4d2",
  from: "0x9f9497c56b54890fd2f693bac26c71e4736eb2be",
  gas: 21000,
  gasPrice: 1000000000,
  hash: "0xe181e503b93f95f2e69553091fbdd8459eda0942662c68ead0c5492779cf31c7",
  input: "0x",
  nonce: 4,
  r: "0xd7bb1c00265f84c46d3c4fbf9ed8228eb791fb0e0303f1d2098989bfed21ecd1",
  s: "0x676eb39f0310ac550aead7bade8f2988b99e64fcbcb743cf91d2d889298c6d9",
  to: "0xfc09dce1ab0fe679e5cc09d88b3e99de13864433",
  transactionIndex: null,
  type: "0x0",
  v: "0x9c7",
  value: 90009000
}
> INFO [03-31|12:19:48.198] Looking for peers      peercount=1 tried=0 static=0
> INFO [03-31|12:19:58.215] Looking for peers      peercount=1 tried=0 static=0
```

11. To check the contents in the Mempool - Transaction Pool

```
> txpool.content
{
  pending: {
    0x9f9497c56b54890fd2f693bac26c71e4736eb2be: {
      0: {
        blockHash: null,
        blockNumber: null,
        chainId: "0x4d2",
        from: "0x9f9497c56b54890fd2f693bac26c71e4736eb2be",
        gas: "0x5208",
        gasPrice: "0x3b9aca00",
        hash: "0xf2c1e10c56397e2726e94bc166405b3ff0863de4757b479a1b6812debe56f33f",
        input: "0x",
        nonce: "0x0",
        r: "0xfe80302530833e03cb38f654004a63d2b812eeab0d15ac0c743c02959e32b17e",
        s: "0x532a68167bf51184d46643bc35d828d941fc9a75a043b4b37010a7409cb41f1c",
        to: "0xfc09dce1ab0fe679e5cc09d88b3e99de13864433",
        transactionIndex: null,
        type: "0x0",
        v: "0x9c8",
        value: "0x0"
      },
      1: {
        blockHash: null,
        blockNumber: null,
        chainId: "0x4d2",
        from: "0x9f9497c56b54890fd2f693bac26c71e4736eb2be",
        gas: "0x5208",
        gasPrice: "0x3b9aca00",
        hash: "0xdde43d6b52e779b78d33a4e5c311ce24b71fc7c7434c93464d662ce20f81ec3c",
        input: "0x",
        nonce: "0x1",
        r: "0xa1abb8a85a39d5da37ac14f8275a2eb5034e8182bd4cfa7b3523d631020188f3",
        s: "0xfb96f3fb07299f86289f4a30805b0872a36b091f9c9c65e577de70adb609e11",
        to: "0xfc09dce1ab0fe679e5cc09d88b3e99de13864433",
        transactionIndex: null,

```

12. To check the status of the Mempool - Transaction Pool

```
> txpool.status
{
  pending: 5,
  queued: 0
}
> INFO [03-31|12:21:38.389] Looking for peers                peercount=1 tried=0 static=0
```

Conclusion:-

This experiment successfully demonstrated the setup of a Private Ethereum Blockchain using Geth with the Clique (PoA) consensus mechanism. By configuring a custom genesis block and initializing an isolated network, developers were able to test transactions and smart contracts without incurring gas costs. The exercise highlighted how authorized signers can validate blocks efficiently, ensuring secure and controlled operations. Overall, the private network provided a reliable environment for enterprise-grade DApp development and experimentation, free from the computational overhead of public Ethereum networks.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 10

Aim: Explore the Cryptocurrency Landscape

Roll No.	12
Name	Palak Chanchlani
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

Name: Palak Chanchlani
Class: D20A
Roll No: 12

EXPERIMENT 10

Aim:

To explore the cryptocurrency landscape using real-world data and blockchain concepts.

Theory:

This study analyzes the crypto market using Chainlink (LINK) as the focus asset. Data was collected from:

- CoinGecko API – Historical price, market cap, and volume (used in Python).
- CoinMarketCap – Price charts and market ranking.
- Crypto.com – Market stats and news updates.

Cryptocurrency Landscape:

- **Definition:** Digital assets based on blockchain enabling secure, decentralized transactions.
- **Types:**
 - Store of Value – Bitcoin, Litecoin
 - Utility Tokens – Ethereum, Chainlink
 - Stablecoins – USDT, USDC
 - Governance Tokens – UNI, AAVE
- **Volatility:** Prices fluctuate due to market trends and regulations.
- **Importance:** Data analysis helps track performance and trends.
- **Advancements:** DeFi, NFTs, Layer-2 scaling, cross-chain systems.

Data Collection & Analysis:

Using CoinGecko API and Pandas:

- Daily Returns – Measures price changes
- Moving Averages (MA7, MA30) – Trend indicators
- Cumulative Returns – Overall growth
- Rolling Volatility – Risk analysis

Visualization:

A dashboard (Matplotlib) showing:

- Price Trend
- Price vs Volume
- Daily Returns Distribution
- Moving Averages
- Cumulative Returns
- Rolling Volatility

Technological Analysis:

- Consensus: PoW, PoS
- Scalability: Layer-2, sharding
- Interoperability: Chainlink CCIP, Polkadot, Cosmos
- Privacy: zk-SNARKs, ring signatures

Chainlink focuses on staking, oracle improvements, and cross-chain capabilities.

Market Trends:

- Increased crypto adoption by businesses
- Growth in crypto wallets
- Higher network activity
- Integration with fintech
- Evolving regulations

Utilizing Api for analysis and visualization : **Code:**

```
import requests

import matplotlib.pyplot as plt

import pandas as pd

# ----- Fetch Data -----

def get_historical_data(coin_id, days='180', currency='usd'):

    url = f'https://api.coingecko.com/api/v3/coins/{coin_id}/market_chart'

    params = {'vs_currency': currency, 'days': days}

    response = requests.get(url, params=params)

    return response.json()

# ----- Dashboard Function
```

```

data = get_historical_data(coin_id, days)

# Create DataFrame
prices_df = pd.DataFrame(data['prices'], columns=['timestamp', 'price'])
volumes_df = pd.DataFrame(data['total_volumes'], columns=['timestamp', 'volume'])

prices_df['timestamp'] = pd.to_datetime(prices_df['timestamp'], unit='ms')
volumes_df['timestamp'] = pd.to_datetime(volumes_df['timestamp'], unit='ms')

df = pd.merge(prices_df, volumes_df, on='timestamp')
df.set_index('timestamp', inplace=True)

# Calculate extra metrics
df['returns'] = df['price'].pct_change()
df['MA7'] = df['price'].rolling(window=7).mean()
df['MA30'] = df['price'].rolling(window=30).mean()
df['cumulative'] = (1 + df['returns']).cumprod()
df['volatility'] = df['returns'].rolling(window=7).std() * 100

# ----- Plot Dashboard -----
fig, axes = plt.subplots(3, 2, figsize=(16, 12))
fig.suptitle(f'{coin_id.capitalize()} Dashboard (Last {days} Days)', fontsize=16, fontweight='bold')

# 1. Price Trend
axes[0, 0].plot(df.index, df['price'], color='blue')
axes[0, 0].set_title('Price Trend')
axes[0, 0].set_ylabel('Price (USD)')
axes[0, 0].set_xlabel('Year-Month') # Add xlabel
axes[0, 0].grid(True)

```

2. Price vs Volume

```
axes[0, 1].plot(df.index, df['price'], color='blue', label='Price')
axes[0, 1].bar(df.index, df['volume'], color='orange', alpha=0.3, label='Volume')
axes[0, 1].set_title('Price vs Volume')
axes[0, 1].set_xlabel('Year-Month') # Add xlabel
axes[0, 1].set_ylabel('Price (USD) / Volume') # Add ylabel
axes[0, 1].legend()
axes[0, 1].grid(True)
```

3. Daily Returns Distribution

```
axes[1, 0].hist(df['returns'].dropna(), bins=50, alpha=0.7, color='purple')
axes[1, 0].set_title('Daily Returns Distribution')
axes[1, 0].set_xlabel('Daily Return')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].grid(True)
```

4. Moving Averages

```
axes[1, 1].plot(df.index, df['price'], color='lightgray', label='Price')
axes[1, 1].plot(df.index, df['MA7'], color='blue', label='7-day MA')
axes[1, 1].plot(df.index, df['MA30'], color='red', label='30-day MA')
axes[1, 1].set_title('Moving Averages')
axes[1, 1].set_xlabel('Year-Month')
axes[1, 1].set_ylabel('Price (USD)')
axes[1, 1].legend()
axes[1, 1].grid(True)
```

5. Cumulative Returns

```
axes[2, 0].plot(df.index, df['cumulative'], color='green')
axes[2, 0].set_title('Cumulative Returns')
axes[2, 0].set_ylabel('Growth Factor')
```



```
axes[2, 0].set_xlabel('Year-Month')
```

```
axes[2, 0].grid(True)
```

```
# 6. Volatility
```

```
axes[2, 1].plot(df.index, df['volatility'], color='orange')
```

```
axes[2, 1].set_title('Rolling Volatility (7-Day)')
```

```
axes[2, 1].set_ylabel('Volatility (%)')
```

```
axes[2, 1].set_xlabel('Year-Month')
```

```
axes[2, 1].grid(True)
```

```
plt.tight_layout(rect=[0, 0, 1, 0.96])
```

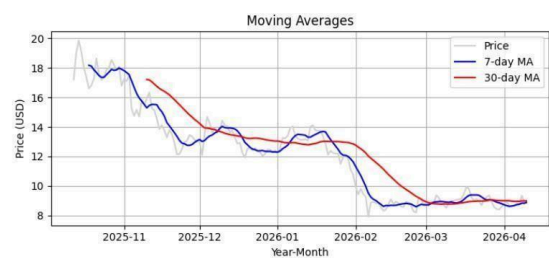
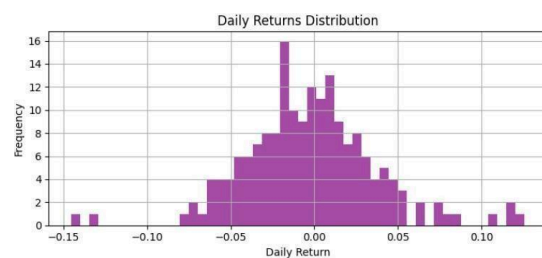
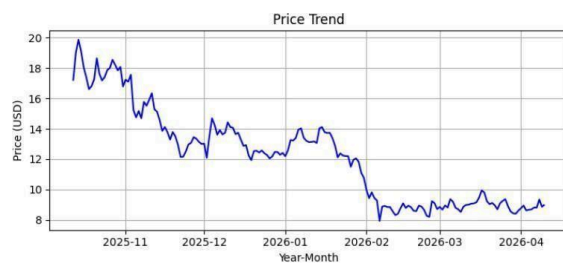
```
plt.subplots_adjust(hspace=0.5, wspace=0.3) # Adjust vertical and horizontal spacing
```

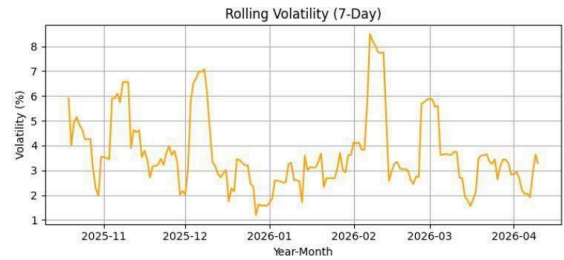
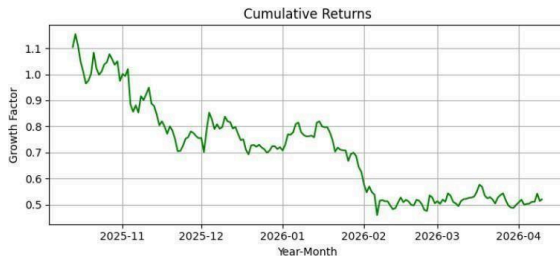
```
plt.show()
```

```
# ----- Run Dashboard
```

```
-----chainlink_dashboard('180')
```

Output:





Coingecko Website

coingecko.com

Now LIVE: Spot CEX Report 2026

Coins: 17,728 Exchanges: 1,457 Market Cap: \$2,527T ▲ 0.9% 24h Vol: \$87,101B Dominance: BTC 57.0% ETH 10.6% Gas: 0.126 GWEI

coingecko Cryptocurrencies Exchanges NFT Learn Products API Candy Portfolio Search

GET 5 BTC FOR FREE +100 **PLAY NOW**

Cryptocurrency Prices by Market Cap

The global cryptocurrency market cap today is \$2.53 Trillion, a ▲ 0.9% change in the last 24 hours. [Read more](#)

Highlights ☒

Market Cap	24h Trading Volume	Trending	Top Gainers
\$2,526,881,227,609 ▲ 0.9%	\$87,100,760,179	- Zcash: \$363.04 ▲ 11.1% - Bitcoin: \$71,924.45 ▲ 1.1% - Siren: \$0.7399 ▲ 29.8%	- RaveDAO: \$0.5855 ▲ 92.1% - Xphere: \$0.01634 ▲ 63.7% - Siren: \$0.7399 ▲ 29.8%

coingecko.com/en/coins/bitcoin

Bitcoin BTC Price #1

\$72,037.64 ▲ 1.2% (24h)

1.0000 BTC ▲ 0.0%

24h Range: \$70,522.33 - \$72,495.25

Add to Portfolio • 2,367,265 added

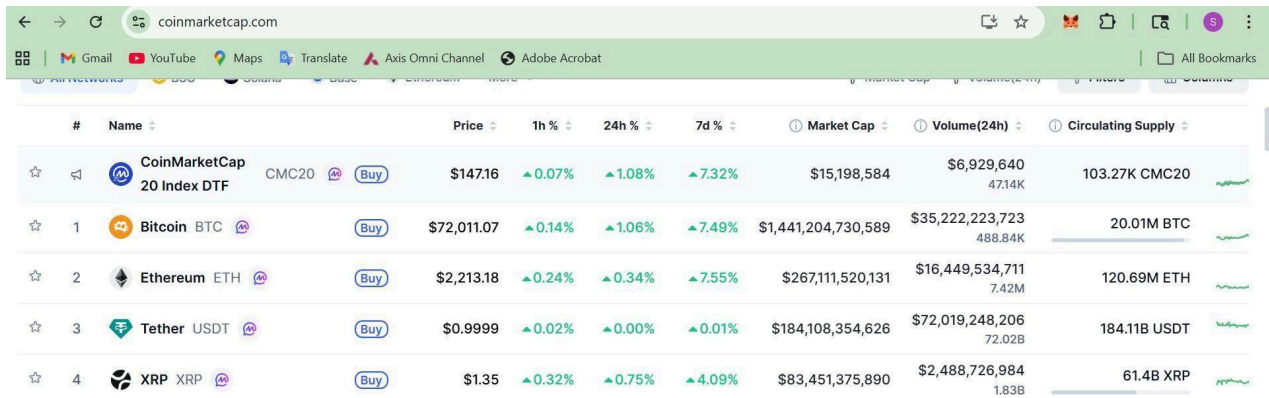
Metric	Value
Market Cap	\$1,441,734,490,650
Fully Diluted Valuation	\$1,441,734,490,650
24 Hour Trading Vol	\$35,707,938,832
Circulating Supply	20,013,637
Total Supply	20,013,637
Max Supply	21,000,000

Overview Markets Treasuries News Similar Coins Historical Data BTC Halving

Price Compare 24H 7D 1M 3M YTD 1Y Max

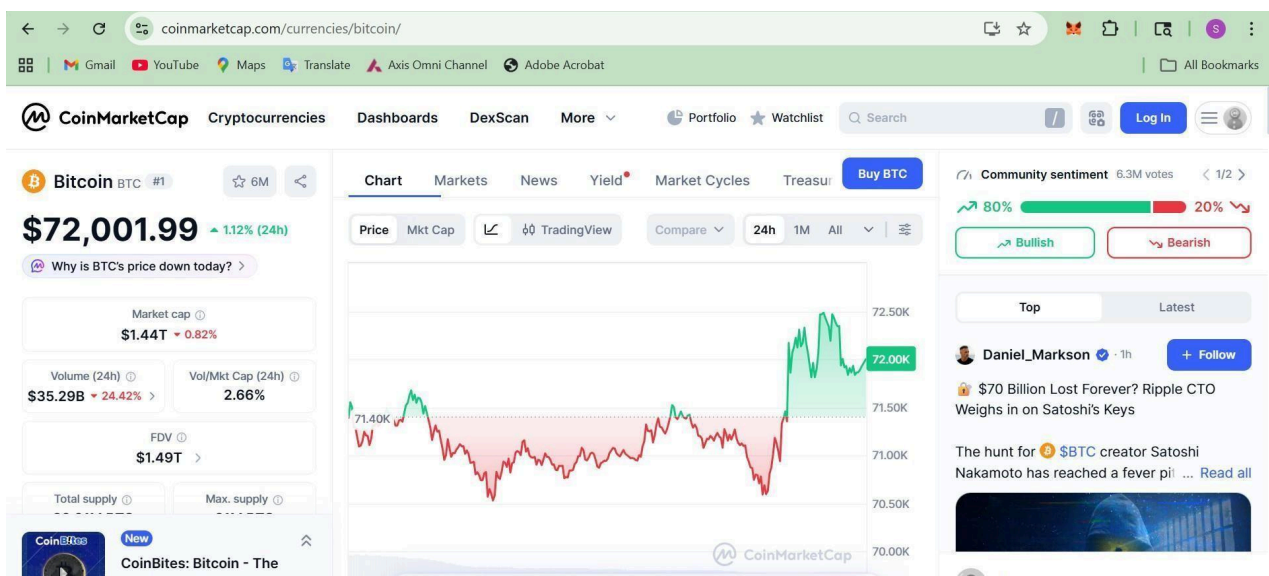
coingecko

CoinMarketCap website



This screenshot shows the main page of the CoinMarketCap website. At the top, there is a navigation bar with the site's logo and various utility links. Below this, a table lists the top cryptocurrencies. Each row includes a rank, the coin's name and symbol, a 'Buy' button, the current price, and percentage changes over 1 hour, 24 hours, and 7 days. Additionally, it displays the market capitalization, 24-hour trading volume, and circulating supply for each coin.

#	Name	Price	1h %	24h %	7d %	Market Cap	Volume(24h)	Circulating Supply
	CoinMarketCap 20 Index DTF	\$147.16	▲0.07%	▲1.08%	▲7.32%	\$15,198,584	\$6,929,640 47.14K	103.27K CMC20
1	Bitcoin BTC	\$72,011.07	▲0.14%	▲1.06%	▲7.49%	\$1,441,204,730,589	\$35,222,223,723 488.84K	20.01M BTC
2	Ethereum ETH	\$2,213.18	▲0.24%	▲0.34%	▲7.55%	\$267,111,520,131	\$16,449,534,711 7.42M	120.69M ETH
3	Tether USDT	\$0.9999	▲0.02%	▲0.00%	▲0.01%	\$184,108,354,626	\$72,019,248,206 72.02B	184.11B USDT
4	XRP XRP	\$1.35	▲0.32%	▲0.75%	▲4.09%	\$83,451,375,890	\$2,488,726,984 1.83B	61.4B XRP



Conclusion:

By combining automated market data processing, visual trend analysis, blockchain technology evaluation, and adoption metrics, this experiment presents a holistic view of the cryptocurrency ecosystem. It demonstrates how blockchain's technical foundations connect with real-world market behavior, providing insights into both current trends and future advancements in the digital asset industry.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 11

Aim: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Roll No.	12
Name	Palak Chanchlani
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO4:Develop applications in permissioned Hyperledger Fabric network.

Name: Palak Chanchlani

Class: D20A

Roll No: 12

EXPERIMENT 11

Aim:

To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

THEORY:

Hyperledger Fabric Overview

Hyperledger Fabric is an open-source, permissioned blockchain framework designed specifically for enterprise applications. Unlike public blockchains where anyone can join, Fabric restricts access to authorized participants, ensuring higher levels of privacy, security, and governance. This makes it particularly suitable for industries such as banking, healthcare, supply chain, and digital identity systems, where confidentiality and trust are critical.

Fabric follows a modular architecture that allows organizations to customize components such as consensus, membership, and data privacy according to their business needs. Its permissioned nature ensures that transactions are only visible to relevant parties, making it a practical solution for enterprise-level deployments.

Key Components of Hyperledger Fabric:

- **Peers:** Nodes that maintain the distributed ledger and execute transactions.
- **Ordering Service:** Ensures transactions are arranged in a consistent sequence before being committed.
- **Channels:** Private communication layers that allow subsets of participants to share confidential data securely.
- **Chaincode:** Business logic of the application, similar to smart contracts, written in Go, Java, or JavaScript.
- **Ledger:** Consists of two parts — the immutable blockchain (record of all transactions) and the world state (latest asset values for quick access).

Chaincode and Transactions

Chaincode defines how assets are created, updated, transferred, and queried. When a transaction is initiated, endorsing peers execute the chaincode, and the results are validated before being recorded on the ledger. This ensures both accuracy and trust in the system.

Features of Hyperledger Fabric

- **Permissioned Access:** Only authorized users can participate.
- **Privacy & Confidentiality:** Channels ensure sensitive data is shared only among relevant parties.
- **Transparency & Immutability:** All transactions are permanently recorded on the blockchain.
- **Scalability & Performance:** Modular design supports high throughput and enterprise workloads.
- **Flexibility:** Supports multiple programming languages for chaincode development.

Applications of Hyperledger Fabric

Hyperledger Fabric is widely adopted in:

- **Supply Chain Management:** Tracking goods and verifying authenticity.
- **Banking & Finance:** Secure transaction processing and compliance.
- **Healthcare:** Managing patient records with privacy.
- **Digital Identity Systems:** Providing secure and verifiable identity management.

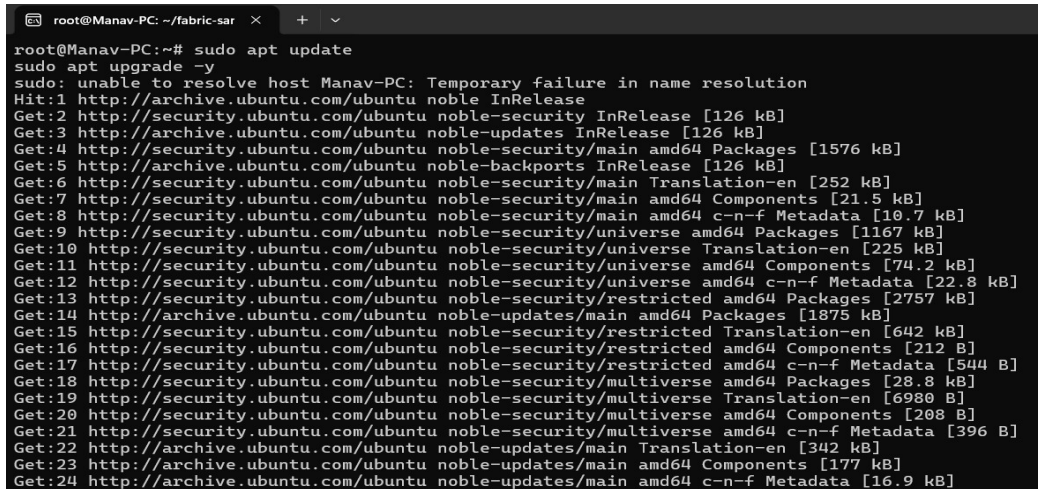
Procedure:

Step 1: Update the System

Command:

```
sudo apt update
```

```
sudo apt upgrade -y
```



```
root@Manav-PC: ~/fabric-sar
root@Manav-PC:~# sudo apt update
sudo apt upgrade -y
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1576 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [252 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [10.7 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1167 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [225 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [22.8 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2757 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1875 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [642 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:17 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 c-n-f Metadata [544 B]
Get:18 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [28.8 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [6980 B]
Get:20 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]
Get:21 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata [396 B]
Get:22 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [342 kB]
Get:23 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:24 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.9 kB]
```

Explanation:

This step updates the system package index and upgrades all installed packages to their latest versions. It ensures that the system has the most recent security patches and compatible libraries required for Hyperledger Fabric. Running this step avoids dependency conflicts and installation failures later.

Step 2: Install Required Dependencies

Command:

```
sudo apt install jq curl git nodejs npm -y
```

```
node -v
```

```
npm -v
```

```
git --version
```

```
docker --
```

```
version docker
```

```
ps
```

```

root@Manav-PC: ~/fabric-sar
root@Manav-PC:~# node -v
v18.19.1
root@Manav-PC:~# npm -v
9.2.0
root@Manav-PC:~# git --version
git version 2.43.0
root@Manav-PC:~# docker --version
Command 'docker' not found, but can be installed with:
snap install docker          # version 28.1.1+1, or
apt install docker.io        # version 28.2.2-0ubuntu1~24.04.
apt install podman-docker    # version 4.9.3+ds1-1ubuntu0.2
See 'snap info docker' for additional versions.
root@Manav-PC:~# docker ps
Command 'docker' not found, but can be installed with:
snap install docker          # version 28.1.1+1, or
apt install docker.io        # version 28.2.2-0ubuntu1~24.04.
apt install podman-docker    # version 4.9.3+ds1-1ubuntu0.2
See 'snap info docker' for additional versions.

```

Explanation:

This step installs all necessary tools required to run Hyperledger Fabric. Docker is used to run blockchain components as containers, Git is used to clone repositories, and Node.js with npm is required for chaincode execution. Version checks confirm successful installation, while `docker ps` ensures Docker is running properly.

Step 3: Clone Fabric Samples

Command:

```
git clone https://github.com/hyperledger/fabric-
samples cd fabric-samples
```

```

root@Manav-PC:~# git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (216/216), done.
remote: Compressing objects: 100% (124/124), done.
remote: Total 15618 (delta 144), reused 92 (delta 92), pack-reused 15402 (from 3)
Receiving objects: 100% (15618/15618), 24.07 MiB | 4.02 MiB/s, done.
Resolving deltas: 100% (8809/8809), done.
root@Manav-PC:~# cd fabric-samples
root@Manav-PC:~/fabric-samples# curl -sSLO https://raw.githubusercontent.com/hyper
ric.sh
chmod +x install-fabric.sh

```


Explanation:

This step downloads the official Hyperledger Fabric sample repository. It contains pre-built network configurations, scripts, and example chaincodes. Moving into this directory allows us to access all required files for setting up the blockchain network.

Step 4: Install Fabric Binaries and Images

Command:

```
curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker
```

```
root@Manav-PC:~/fabric-samples# curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
root@Manav-PC:~/fabric-samples# chmod +x install-fabric.sh
root@Manav-PC:~/fabric-samples# ./install-fabric.sh docker samples binary
root@Manav-PC:~/fabric-samples# ./install-fabric.sh docker

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is the latest version of fabric.

Pull Hyperledger Fabric binaries

==> Downloading version 2.5.15 platform specific fabric binaries
==> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-v2.5.15-linux-amd64.tar.gz
==> Will unpack to: /root/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left     Speed
  0     0    0     0    0     0      0      0  --:--:-- --:--:-- --:--:--     0
100 120M  100 120M    0     0 3701k      0  0:00:33  0:00:33 --:--:-- 3516k
==> Done.
```

Pull Hyperledger Fabric docker images

```
FABRIC_IMAGES: peer orderer ccenv baseos
====> Pulling fabric Images
====> ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
Digest: sha256:3e25adc31a757ee19dd8a24afdc9ac5aa46a86b3cb56fc93068b0e594706d6aa
Status: Image is up to date for ghcr.io/hyperledger/fabric-peer:2.5.15
ghcr.io/hyperledger/fabric-peer:2.5.15
====> ghcr.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
Digest: sha256:9972c209be47f45e377a24c88f2e3d21fae4dc224751c68bdd33f2e7a603ffa8
Status: Image is up to date for ghcr.io/hyperledger/fabric-orderer:2.5.15
ghcr.io/hyperledger/fabric-orderer:2.5.15
====> ghcr.io/hyperledger/fabric-ccenv:2.5.15
2.5.15: Pulling from hyperledger/fabric-ccenv
Digest: sha256:86dded7eda9268e2cbf3691cb952edf545dc5faea1a79bfc9b47ec47d3ecfa9e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ccenv:2.5.15
ghcr.io/hyperledger/fabric-ccenv:2.5.15
====> ghcr.io/hyperledger/fabric-baseos:2.5.15
2.5.15: Pulling from hyperledger/fabric-baseos
Digest: sha256:654bd4554bf97c70902e56d530294795372518da7ae2a7cdfccbe397a878c513
Status: Image is up to date for ghcr.io/hyperledger/fabric-baseos:2.5.15
ghcr.io/hyperledger/fabric-baseos:2.5.15
====> Pulling fabric ca Image
====> ghcr.io/hyperledger/fabric-ca:1.5.17
```

Explanation:

This step downloads and executes a script that installs Hyperledger Fabric binaries and required Docker images. It pulls essential components like peer, orderer, and certificate authority images. These components are necessary to create and manage the blockchain network.

Step 5: Verify Docker Images

Command:

docker images | grep fabric

```
root@Manav-PC:~/fabric-samples# docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output,
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17          453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15       86dded7eda92      1GB       255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15     9972c209be47      178MB     49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15        3e25adc31a75      230MB     66.2MB
hyperledger/fabric-baseos:2.5                 654bd4554bf9      250MB     80.4MB
hyperledger/fabric-baseos:2.5.15              654bd4554bf9      250MB     80.4MB
hyperledger/fabric-baseos:latest              654bd4554bf9      250MB     80.4MB
hyperledger/fabric-ca:1.5                     453a0a727e82      381MB     123MB
hyperledger/fabric-ca:1.5.17                  453a0a727e82      381MB     123MB
hyperledger/fabric-ca:latest                  453a0a727e82      381MB     123MB
hyperledger/fabric-ccenv:2.5                  86dded7eda92      1GB       255MB
hyperledger/fabric-ccenv:2.5.15               86dded7eda92      1GB       255MB
hyperledger/fabric-ccenv:latest               86dded7eda92      1GB       255MB
hyperledger/fabric-orderer:2.5                9972c209be47      178MB     49.5MB
hyperledger/fabric-orderer:2.5.15             9972c209be47      178MB     49.5MB
hyperledger/fabric-orderer:latest             9972c209be47      178MB     49.5MB
hyperledger/fabric-peer:2.5                   3e25adc31a75      230MB     66.2MB
hyperledger/fabric-peer:2.5.15                3e25adc31a75      230MB     66.2MB
hyperledger/fabric-peer:latest                3e25adc31a75      230MB     66.2MB
```

Explanation:

This command checks whether all required Hyperledger Fabric Docker images are successfully downloaded. It ensures that components such as peer and orderer are available before starting the network. Missing images may cause deployment errors.

Step 6: Set Environment Variables**Command:**

```
export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-
samples/config peer version
```

```
root@Manav-PC:~/fabric-samples# export PATH=$PATH:$HOME/fabric-samples/bin
root@Manav-PC:~/fabric-samples# export FABRIC_CFG_PATH=$HOME/fabric-samples/config
root@Manav-PC:~/fabric-samples# peer version
peer:
  Version: v2.5.15
  Commit SHA: 83c7930
  Go version: go1.26.0
  OS/Arch: linux/amd64
  Chaincode:
    Base Docker Label: org.hyperledger.fabric
    Docker Namespace: hyperledger
```

Explanation:

This step sets environment variables required to run Fabric CLI commands. The PATH variable allows access to Fabric binaries, while FABRIC_CFG_PATH points to configuration files.

Running `peer version` verifies that the setup is successful.

Step 7: Start Network and Create Channel**Command:**

```
cd ~/fabric-samples/test-network
./network.sh up createChannel
```

```

root@Manav-PC:~/fabric-samples# cd ~/fabric-samples/test-network
./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using
ldb with crypto from 'cryptogen'
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/root/fabric-samples/test-network/./bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] up 7/7
✓Network fabric_test Created
✓Volume compose_orderer.example.com Created

```

```

root@Manav-PC: ~/fabric-sar  X + v
+ peer lifecycle chaincode commit -o localhost:7050 --ordererTLSHostnameOverride orde
t/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tl
D mychannel --name basic --peerAddresses localhost:7051 --tlsRootCertFiles /root/fabr
ns/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem --peerAdd
Files /root/fabric-samples/test-network/organizations/peerOrganizations/org2.example.
rt.pem --version 1.0 --sequence 1
+ res=0
2026-04-03 13:22:50.210 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [136f63476f31
689af6a0c344c441b] committed with status (VALID) at localhost:9051
2026-04-03 13:22:50.215 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [136f63476f31
689af6a0c344c441b] committed with status (VALID) at localhost:7051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approva
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0

```

Explanation:

This step initializes the blockchain network by starting Docker containers for peers and orderers. It also creates a communication channel where transactions will occur. Certificates and configurations are automatically generated during this process.

Step 8: Deploy Chaincode

Command:

```
./network.sh deployCC \  
-ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript \  
-ccl javascript
```

```
root@Manav-PC:~/fabric-samples# cd ~/fabric-samples/test-network  
root@Manav-PC:~/fabric-samples/test-network# ./network.sh deployCC \  
-ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript \  
-ccl javascript  
Using docker and docker-compose  
deploying chaincode on channel 'mychannel'  
executing with the following  
- CHANNEL_NAME: mychannel  
- CC_NAME: basic  
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript  
- CC_SRC_LANGUAGE: javascript  
- CC_VERSION: 1.0  
- CC_SEQUENCE: auto  
- CC_END_POLICY: NA  
- CC_COLL_CONFIG: NA  
- CC_INIT_FCN: NA  
- DELAY: 3  
- MAX_RETRY: 5  
- VERBOSE: false  
executing with the following  
- CC_NAME: basic  
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript  
- CC_SRC_LANGUAGE: javascript  
- CC_VERSION: 1.0  
+ '[' false = true ']'  
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-bas  
basic_1.0  
+ res=0  
Chaincode is packaged
```

Explanation:

This step deploys the smart contract (chaincode) onto the blockchain network. The chaincode defines the business logic for asset operations such as creation, transfer, and querying. Once deployed, it is ready to interact with the ledger.

Step 9: Set Peer Environment

Command:

```
export FABRIC_CFG_PATH=$HOME/fabric-  
samples/config export PATH=$PATH:$HOME/fabric-  
samples/bin  
export CORE_PEER_TLS_ENABLED=true  
export  
CORE_PEER_LOCALMSPID="Org1MSP"  
export  
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-  
network/organizations/peer  
Organizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt  
export  
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-  
network/organizations/peerOrg  
anizations/org1.example.com/users/Admin@org1.example.com/msp  
export CORE_PEER_ADDRESS=localhost:7051
```

```
root@Manav-PC:~/fabric-samples/test-network# export FABRIC_CFG_PATH=$HOME/fabric-samples/config  
export PATH=$PATH:$HOME/fabric-samples/bin  
export CORE_PEER_TLS_ENABLED=true  
export CORE_PEER_LOCALMSPID="Org1MSP"  
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/  
ers/peer0.org1.example.com/tls/ca.crt  
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1  
Admin@org1.example.com/msp  
export CORE_PEER_ADDRESS=localhost:7051
```

Explanation:

This step configures the identity and communication settings of the peer node. It specifies organization details, TLS certificates, and network addresses. These settings are required for secure interaction with the blockchain network.

Step 10: Initialize Ledger

Command:

peer chaincode invoke ... InitLedger

```
root@Manav-PC:~/fabric-samples/test-network# peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer  
/msp/tlscacerts/tlsca.example.com-cert.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/pee  
.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/pee  
.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function":"InitLedger","Args":[]}'  
2026-04-03 13:27:01.786 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [739a6ae87df069979d1bc20a6341876226  
c12b3bd8f2591250b] committed with status (VALID) at localhost:7051  
2026-04-03 13:27:01.787 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [739a6ae87df069979d1bc20a6341876226  
c12b3bd8f2591250b] committed with status (VALID) at localhost:9051  
2026-04-03 13:27:01.790 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful.  
ms:200
```

Explanation:

This step initializes the blockchain ledger with default sample assets. It writes initial data to the ledger, which can later be queried or modified. This operation is a transaction that gets recorded on the blockchain.

Step 11: Query All Assets**Command:**

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

```
root@Manav-PC:~/fabric-samples/test-network# peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

Explanation:

This step retrieves all assets stored in the ledger. It performs a read-only operation and displays current data without modifying the blockchain. It helps verify that the ledger has been initialized correctly.

Step 12: Read Specific Asset**Command:**

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
```

```
root@Manav-PC:~/fabric-samples/test-network# peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
```

Explanation:

This step retrieves detailed information of a specific asset using its ID. It demonstrates how individual records can be accessed from the ledger. This is also a read-only operation.

Step 13: Create New Asset

Command:

peer chaincode invoke ... CreateAsset (Owner: Manav)

```
root@Manav-PC:~/fabric-samples/test-network# peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com
/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1
.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2
.example.com/tls/ca.crt \
--waitForEvent \
-c '{"function":"CreateAsset","Args":["asset10","blue","10","Manav","1000"]}'
2026-04-03 13:37:44.489 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [c0fb2f13a0c62f343bed8b6ae7e7fc02db99ba8f961f539
2ad0df3b16321450e] committed with status (VALID) at localhost:7051
2026-04-03 13:37:44.490 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [c0fb2f13a0c62f343bed8b6ae7e7fc02db99ba8f961f539
2ad0df3b16321450e] committed with status (VALID) at localhost:9051
2026-04-03 13:37:44.490 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: stat
us:200 payload:{"ID":"asset10","Color":"blue","Size":10,"Owner":"Manav","AppraisedValue":1000}
```

Explanation:

This step creates a new asset on the blockchain with specified attributes such as ID, color, size, owner, and value. The owner name is set to Manav, which confirms that the transaction was performed by the user. This operation updates the ledger.

Step 14: Transfer Asset

Command:

peer chaincode invoke ... TransferAsset

```
root@Manav-PC:~/fabric-samples/test-network# peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com
/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1
.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2
.example.com/tls/ca.crt \
--waitForEvent \
-c '{"function":"TransferAsset","Args":["asset10","Rahul"]}'
2026-04-03 13:41:46.421 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [223f359d1497e07db6d93c10e9a6c2917665628376992f7
baca5ad0d3aaadd] committed with status (VALID) at localhost:9051
2026-04-03 13:41:46.424 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [223f359d1497e07db6d93c10e9a6c2917665628376992f7
baca5ad0d3aaadd] committed with status (VALID) at localhost:7051
2026-04-03 13:41:46.426 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: stat
us:200 payload:"Manav"
```


Explanation:

This step transfers ownership of an asset from one user to another. It updates the owner field in the ledger. This demonstrates how blockchain ensures secure and traceable asset transfers.

Step 15: Stop Network**Command:**

`./network.sh down`

```
root@Manav-PC:~/fabric-samples/test-network# ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
✓Container peer0.org1.example.com      Removed
✓Container peer0.org2.example.com      Removed
✓Container orderer.example.com         Removed
✓Volume compose_orderer.example.com    Removed
✓Network fabric_test                  Removed
✓Volume compose_peer0.org1.example.com Removed
✓Volume compose_peer0.org2.example.com Removed
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
6152d81175c6
f5b73b9b7204
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f5
2a0ff0cc1497615f5e33a8bfa908e0fd3555f864cfa1398b52d341cb:latest
Deleted: sha256:54e17cd9d88b065838b9b4184da44258c6c5402e7b2aa9ee3a324fc09cdd84fd
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f5
4b9747cc8433ac0e74bfaade1cf0b97f5c1163835f604aa885d60dc:latest
Deleted: sha256:4b8656488ea2c9620f5669d82c6bee43ad10e6a71bb5f57cde69bfdb57fcd860
```

Explanation:

This step stops all running Docker containers and shuts down the Hyperledger Fabric network. It frees system resources and safely ends the experiment.

Conclusion:

This experiment successfully demonstrated the working of blockchain networks using Hyperledger Fabric. It showed how transactions are securely executed in a permissioned environment and how chaincode enables real-time asset management. The exercise highlighted Fabric's modular architecture, privacy features, and efficient transaction validation, proving its suitability for enterprise applications where confidentiality, trust, and performance are essential.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain and DLT Lab
Experiment 12

Aim: Mini Project

Roll No.	12
Name	Palak Chanchlani
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO4:Develop applications in permissioned Hyperledger Fabric network. CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

Blockchain-Based Academic Certificate Verification System

Palak Chanchlani - 12

Kunal Punjabi - 46

Manav Punjabi - 47

Devansh Wadhvani - 62



01

Section 1: Problem Context & Current State

Blockchain-Based Academic Certificates

Team Members & Course Information

[Your Team Names] presents this project for
[Course Name] at [Institution] on
[Submission Date]

Solution Framework Cycle

Blockchain enables immutable record-keeping, instant verification, reduced fraud, and enhanced trust in academic credentials permanently.



Distributed Ledger Innovation

Revolutionizing credential authentication through blockchain technology for secure academic certificate verification and validation systems.

Problem Context Integration

Current certificate systems lack transparency, security, and efficiency in verification processes across educational institutions globally.

Background & Introduction

Domain & Core Challenge

Academic certificates face fraud risks; traditional verification systems are fragmented, time-consuming, lacking universal standardization across institutions globally.

Stakeholder Ecosystem

Educational institutions, employers, verification services, students, and regulatory bodies require seamless collaboration for effective credential management solutions.

Market Demand & Impact

Growing need for instant, tamper-proof verification reducing fraud and processing time from weeks to seconds in competitive global employment landscape.

Problem Statement

01

Core Problem Definition

Current academic credential verification lacks real-time authenticity assurance; manual processes prove inefficient and susceptible to forgery risks.

02

Key Challenges

Multiple disconnected databases lack interoperability; high administrative burden, verification delays average seven to fourteen days regularly.

04

Vulnerability & Fraud Risk

Systems remain vulnerable to document tampering and credential fraud; verification agencies struggle with disconnected institutional databases.

03

Primary Stakeholders

Educational institutions, employers, students, regulatory authorities, and third-party verification agencies require efficient credential verification solutions.

Existing System Analysis

Current Process

Paper-based or centralized digital records maintained independently by each institution without standardized format across systems.

Key Limitations

Vulnerable to fraud and document manipulation with lack of real-time verification capability across institutions globally.



Root Cause

Absence of unified, immutable, transparent verification infrastructure; siloed institutional systems prevent cross-institutional verification.

Stakeholder Pain Points

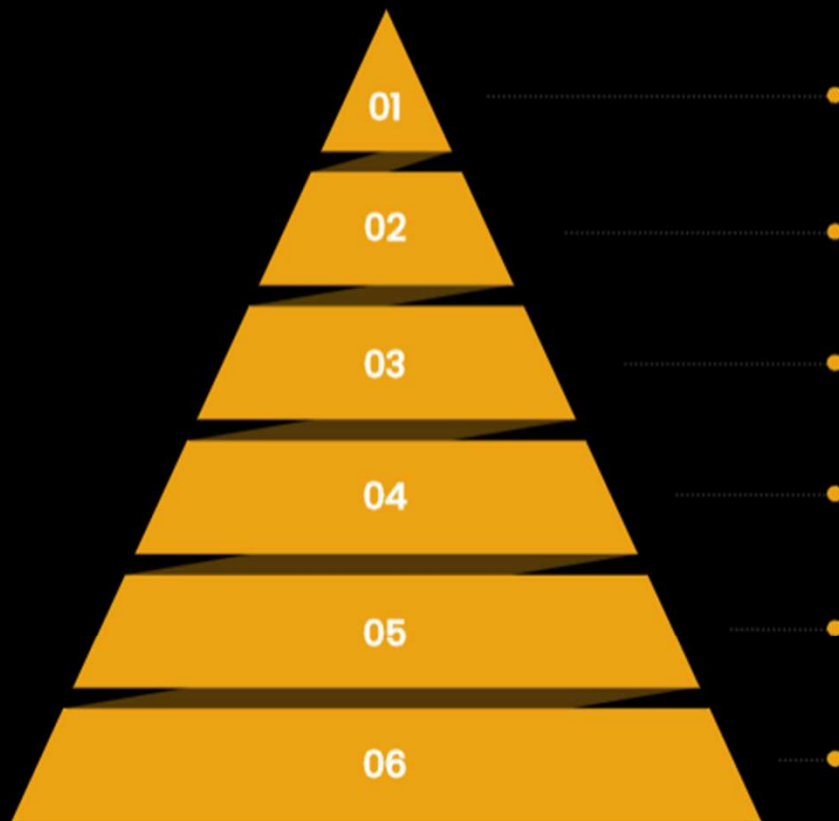
Employers face trust issues; students experience verification delays; institutions bear high administrative costs for manual processes.



02

Section 2: Blockchain Solution & Architecture

Why Blockchain?



Digital Transformation Imperative

Education requires trustless, immutable verification infrastructure for modern credential management and stakeholder confidence.

Key Benefits Layer

Blockchain enables decentralized ledger, cryptographic security, and real-time verification without intermediaries or delays.

Immutability Advantage

Once recorded, certificates cannot be altered, ensuring permanent integrity and preventing fraudulent modifications.

Transparency Benefit

All stakeholders verify authenticity instantly, enabling immediate credential validation across global institutions and employers.

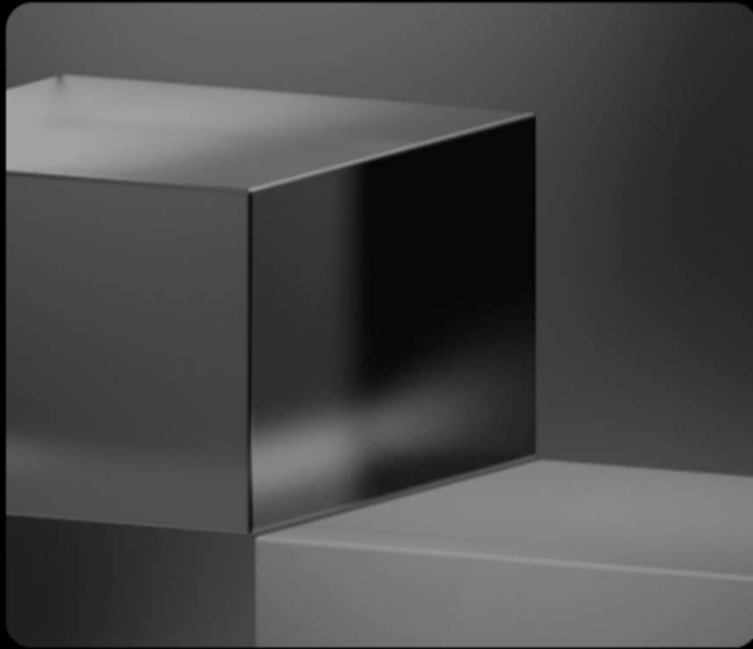
Efficiency Gain

Eliminates intermediaries and reduces verification time to seconds, streamlining processes and lowering operational costs.

Security Foundation

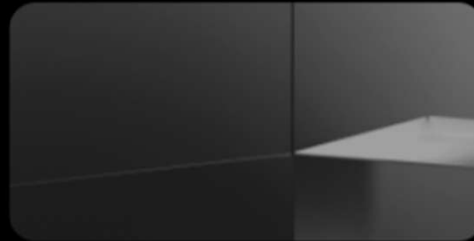
Cryptographic hashing prevents fraud and forgery, protecting certificates with military-grade encryption and authentication mechanisms.

System Architecture



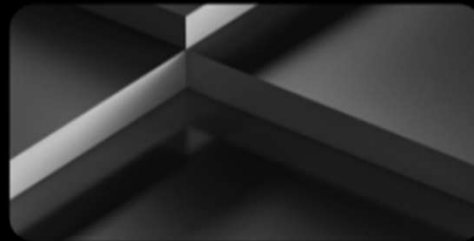
Network Components

Educational institutions, employers, regulatory bodies, and student digital wallets form the distributed network nodes managing certificate verification operations.



Smart Contracts

Automated systems handle certificate issuance, verification, and revocation processes through programmed logic ensuring consistency and security.



Data Layer

Distributed ledger maintains certificate hashes, metadata, and issuer signatures enabling immutable record verification and authenticity.



Consensus Mechanism

Practical Byzantine Fault Tolerance (PBFT) ensures agreement across permissioned network nodes maintaining data integrity and trust.



External Integrations

Student management systems, identity APIs, employer portals, and mobile wallets connect seamlessly enabling comprehensive ecosystem participation.

Working Mechanism

01

Certificate Creation

Institution generates digital certificate containing student credentials, program details, and graduation date information.



02

Cryptographic Hashing

System creates unique hash fingerprint of certificate data to prevent any tampering or unauthorized modifications.



03

Verification

Employer scans certificate QR code or enters certificate ID; system instantly returns verification status results.



04

Smart Contract Invocation

Automated contract verifies issuer credentials and timestamp ensuring legitimacy of certificate issuance process.



05

Immutable Log

Complete audit trail maintained showing issuance, verification attempts, and any revocation actions throughout lifecycle.



06

Blockchain Recording

Certificate record and hash are written to distributed ledger across all nodes for permanent storage.





03

Section 3: Implementation & Impact Analysis

Key Technologies Used

Blockchain Platform

Hyperledger Fabric for enterprise-grade, permissioned networks; Corda for financial institutional use cases and scalability requirements.

Smart Contracts

Solidity and Chaincode automate certificate lifecycle management, enabling secure issuance, validation, and revocation processes efficiently.

Identity Layer

Self-Sovereign Identity standards with decentralized identifiers empower students and institutions with autonomous identity management capabilities.

Integration APIs

RESTful APIs facilitate employer verification portals; OAuth 2.0 ensures secure authentication and authorized credential access seamlessly.

Digital Wallet

Mobile applications store and share credentials securely; QR codes enable instant verification for employers and institutions.

Database Layer

Off-chain storage preserves complete certificate documents; blockchain records only hash and metadata for optimal performance efficiency.

SWOT Analysis

Strengths

Immutable record-keeping ensures certificates cannot be altered or forged. Instant global verification eliminates delays and geographical barriers. Fraud elimination protects institutional reputation. Reduced administrative costs lower operational expenses. Enhanced user trust strengthens system credibility and adoption rates.

Weaknesses

Initial setup and migration costs present significant financial barriers. Resistance from legacy-dependent institutions slows implementation. Technical expertise requirements limit personnel capability. Network adoption dependency means system value increases with participant numbers. Integration challenges with existing infrastructure require substantial resources.

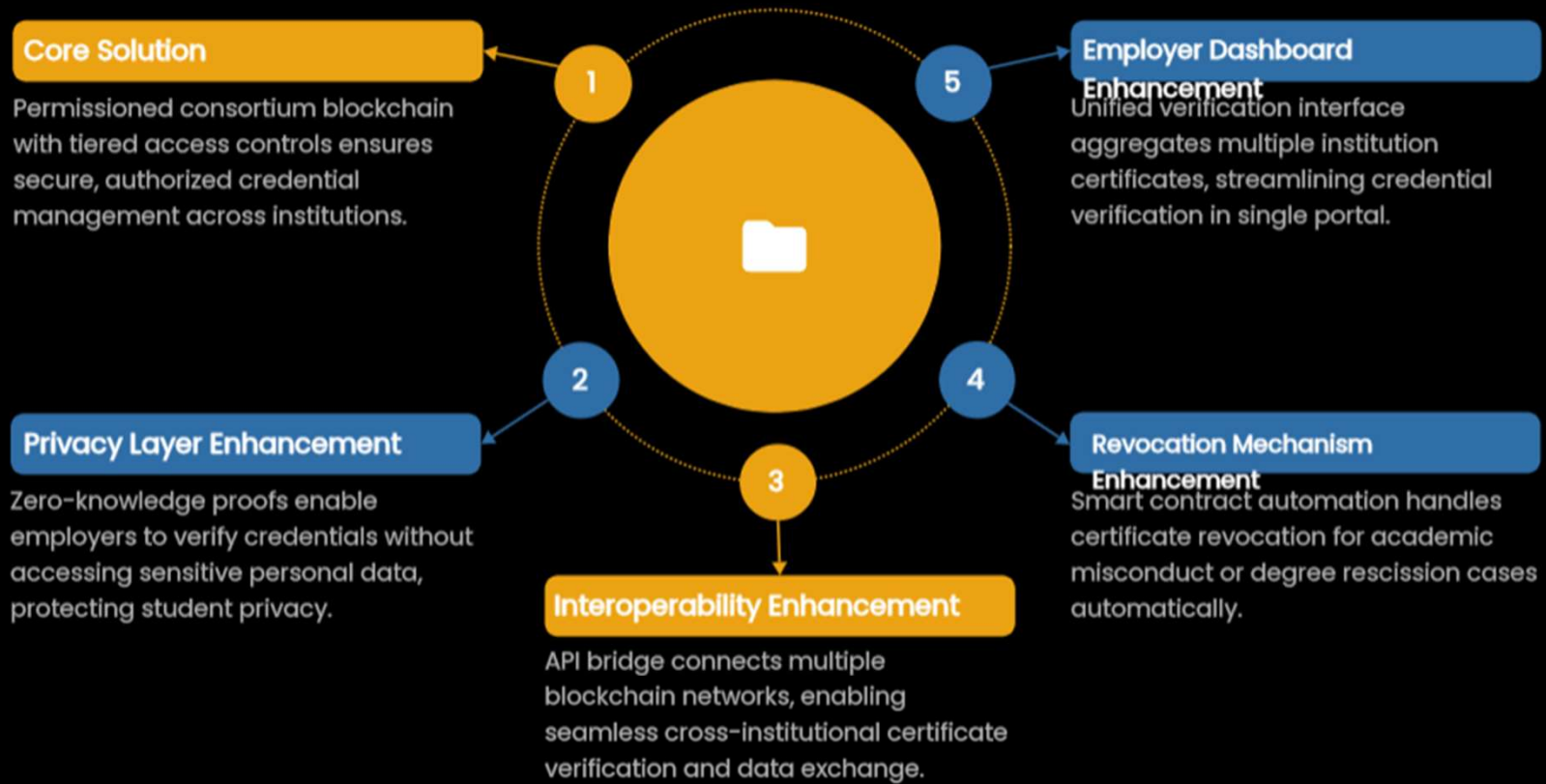
Opportunities

Expansion to other credentials including licenses and certifications broadens market scope. Integration with decentralized identity platforms creates ecosystem synergies. Employer ecosystem partnerships increase practical applications. Emerging markets with limited verification infrastructure offer high-growth opportunities. Cross-industry adoption potential accelerates revenue streams.

Threats

Regulatory uncertainty creates compliance risks across jurisdictions. Competing proprietary verification systems fragment market adoption. Cybersecurity vulnerabilities if improperly implemented expose data risks. Potential privacy concerns with distributed ledger transparency affect user confidence. Technology obsolescence threatens long-term viability.

Proposed Solutions & Enhancements



Recommended Solution

Efficiency Advantage

Three-second verification versus seven to fourteen day manual process achieves ninety-five percent administrative overhead reduction significantly.

Security Assurance

Cryptographic signing prevents forgery while multi-signature validation ensures institutional authentication and document integrity.

Optimal Approach

Hyperledger Fabric-based consortium blockchain with privacy-preserving zero-knowledge proofs implementation for certificate verification.



Transparency & Accountability

Complete audit trail visible to authorized parties with tamper-evident records ensuring accountability and regulatory compliance standards.

Feasibility Analysis

Technical Feasibility

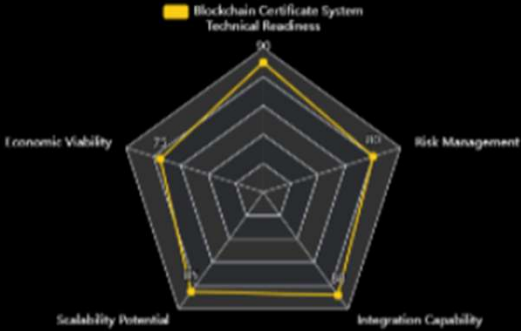
Hyperledger Fabric proven in enterprise environments. API integration straightforward with existing student management systems. High technical readiness for implementation.

Economic Feasibility

Initial setup cost 100-150K. Medium-term ROI within 3-5 years through fraud prevention and reduced administrative overhead.

Scalability Assessment

Current scale 50-100 institutions; target 500+ institutions handling 10M+ certificates annually. Modular architecture enables incremental growth without performance degradation.



Feasibility Dimension	Status	Key Metrics
Technical	High	Hyperledger Fabric proven; API integration straightforward
Economic	Medium-term ROI	\$1M setup; \$100-150K annual costs; 3-5 year payback
Scalability	Excellent	50-100→500+ institutions; 10M+ certificates annually

Implementation Phases

Key work plan for the first half of the year

Jan. → Jun.

Work Plan

Key work plan for the second half of the year

Jul. → Dec.

Design & Planning

Define consortium structure and governance framework. Select appropriate blockchain platform. Conduct comprehensive security audit planning. Establish foundational architecture for system deployment. Duration: Months 1-2.

Development Stage

Build core smart contracts for certificate verification. Develop verification APIs and integration protocols. Create user interfaces for employers and students. Initiate pilot program with 5-10 partner institutions. Duration: Months 3-5.

Testing & Optimization

Execute penetration testing for security assessment. Conduct load testing supporting 10K transactions daily. Perform user acceptance testing with pilot institutions. Validate system reliability and performance metrics. Duration: Months 6-7.

Deployment & Scaling

Implement gradual rollout to additional educational institutions. Establish dedicated production operations center. Monitor continuous system performance and security. Scale infrastructure to support growing network participation. Duration: Months 8+.

Impact Analysis





04

Section 4: Challenges, Limitations & Future Outlook

Challenges & Limitations



Cost Barriers

Initial consortium setup demands significant capital investment; smaller institutions require subsidy models to overcome affordability challenges effectively.



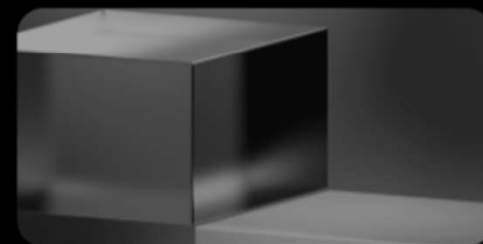
Technical Complexity

Specialized blockchain developers and DevOps expertise remain scarce; emerging markets face acute personnel shortage challenges impacting implementation.



Adoption Barriers

Legacy system integration proves difficult; institutional resistance necessitates industry-wide coordination and change management strategies.



Regulatory Uncertainty

Varying data protection laws across jurisdictions complicate compliance; unclear liability frameworks for blockchain-based records require legal clarification.



Interoperability Issues

Multiple competing blockchain standards risk fragmentation; standardization efforts essential to ensure seamless cross-platform academic certificate verification systems.

Conclusion & Future Scope





Thank
You



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 13

Aim: Assignment 1

Roll No.	12
Name	Palak Chanchlani
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Question : Explore following 4 websites and answer the questions bitcoin.org Blockchain.com https://coinmarketcap.com/ https://www.investopedia.com/
CO Mapped	CO1,CO2

Hard Copy Submitted



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain and DLT Lab
Experiment 14

Aim: Assignment 2

Roll No.	12
Name	Palak Chanchlani
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Questions ● Draw a neat diagram and explain components of Hyperledger Fabric? ● Draw a neat diagram and explain the transaction flow in Hyperledger Fabric? ● State the requirements of a consensus algorithm. Explain PAXOS and kafka consensus ● Explain ERC 20 token standard and its functions and Compare how ERC 20 token is different than ERC 721 token ● What is Tokenization? Explain its significance and challenges/limitation with example ● Compare Fungible and non Fungible tokens (Give examples of each) ● How can blockchain-based IoT networks improve device management, data monetization, and user privacy in the rapidly growing IoT ecosystem? ● In what ways can blockchain technology enhance transparency, security, and efficiency in government services and record-keeping? Discuss the challenges and benefits of implementing blockchain-based voting systems for elections.
CO Mapped	CO4,CO5,CO6

Hard Copy Submitted



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 15

Aim: Assignment 3

Roll No.	12
Name	Palak Chanchlani
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Q1 Analyze & compare different consensus mechanisms such as PoW, PoS, Delegated Proof of Stake & PBFT in terms of security, scalability, energy efficiency & fault tolerance. Illustrate your answer with examples like Bitcoin & Ethereum. Q2 Analyze different types of nodes in a blockchain network & explain their role in maintaining the system. Compare how each node contributes to validation, storage & network efficiency. (Compare UTXO model) Q3 Analyze the UTXO model & explain how it ensures transaction transparency & security in blockchain systems. Illustrate with an example such as Bitcoin. Q4 Explain crypto asset & cryptocurrency. Examine & analyze the role of different types of crypto assets (utility tokens, security tokens, crypto coins) in the blockchain ecosystem. Evaluate their impact on digital finance & decentralized applications. Q5 Analyze the role of payment scripts in blockchain transactions. Examine & analyze different types of payment scripts. Explain how they ensure secure & verifiable transfer of coins. Illustrate with an example. Q6 What is a smart contract? Explain lifecycle of smart contracts. Explain advantages, limitations, risks & opportunities of smart contracts. Q7 Explain the structure of BTC transaction & ETH transaction. Q8 Explain the difference between mining & minting and explain the process
CO Mapped	CO1- CO6

Hard Copy Submitted